

교과목 3 : 초거대언어모델(LLM)

단원 5 : 파인 튜닝

DAY2. PEFT 기반 경량 LLM 파인 튜닝 기법

목 차

1. PEFT 기반 파인 튜닝 기법		3
2. 개발툴에서 RunPod SSH 접속		15
3. Supervised Fine-tuning 실습		51

1. PEFT 기반 파인 튜닝 기법

1. PRFT란 무엇인가?

PEFT(Parameter-Efficient Fine-Tuning) 는 거대 언어 모델(LLM)의 모든 가중치를 학습하지 않고, 일부 작은 파라미터만 학습하여 새로운 작업(Task)에 적응시키는 기술이다.

기존 Fine-tuning 은 수십억 개의 파라미터 전체를 수정한다.

반면 PEFT 는

- 기존 모델은 대부분 그대로 유지(Frozen)
- 작은 Adapter 또는 Low-rank Matrix 만 학습
- GPU 메모리와 학습 시간을 크게 절약

하는 방식이다.

즉, "거대한 모델은 그대로 두고 필요한 부분만 덧붙여 학습한다."라고 이해하면 된다.

2. 기존 Full Fine-tuning 과의 차이

항목	Full Fine-tuning	PEFT
학습 파라미터	100%	0.01~5%
GPU 메모리	매우 큼	매우 작음
학습 속도	느림	빠름
모델 저장	전체 모델	Adapter 만 저장
배포 용량	수 GB~수십 GB	수 MB~수백 MB
여러 Task 지원	어려움	매우 쉬움

예를 들어 Qwen2.5-7B 전체 모델 약 7,000,000,000 Parameter

LoRA Adapter 약 5~20MB 정도만 추가 저장하면 된다.

3. 왜 PRFT가 등장했는가?

GPT, Llama, Qwen 같은 모델은

7B
13B
32B
70B
405B

규모이다.

예를 들어 Llama3 70B 를 Full Fine-tuning 하면
대략 GPU Memory 300GB 이상 필요하다.

하지만 대부분의 개발자는

RTX3060
RTX4070
RTX4090
RunPod A40
A100

정도만 사용한다.

그래서 모델 전체를 수정하지 않고 필요한 부분만 수정하자 -> PEFT 등장

4. PRFT의 핵심 아이디어

기존 Weight

W

를 직접 수정하지 않는다.

대신

$W + \Delta W$

형태로 사용한다.

여기서

W

는 고정(Frozen)

ΔW

만 학습한다.

즉

Original Weight

↓

Freeze

↓

Adapter 만 학습

5. 대표적인 PRFT 기법

현재 가장 많이 사용되는 기법은 다음과 같다.

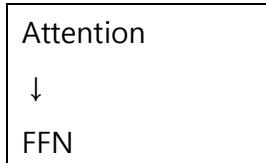
기법	특징
Adapter Tuning	Layer 사이에 작은 네트워크 추가
LoRA	가장 널리 사용
QLoRA	LoRA + 4bit 양자화
Prefix Tuning	Prefix Vector 학습
Prompt Tuning	Virtual Prompt 학습
P-Tuning v2	Deep Prompt
IA3	Attention Scaling
AdaLoRA	Adaptive Rank
DoRA	LoRA 개선
VeRA	Vector Adapter
OFT	Orthogonal Fine-tuning

1. Adapter Tuning

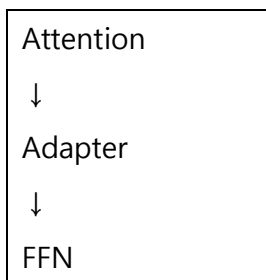
초기의 대표적인 방법이다.

Transformer Block

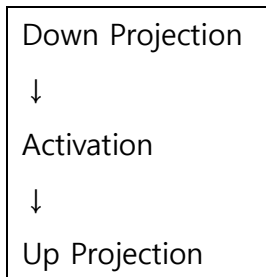
기존



변경



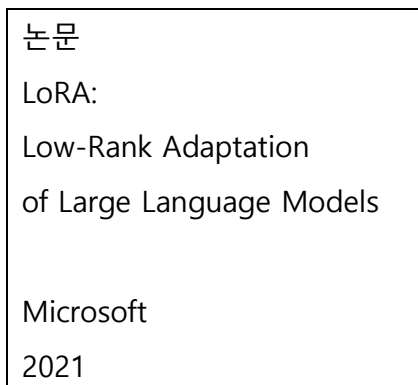
Adapter 는



구조이다. 작은 MLP 만 학습한다.

2. LoRA (Low-Rank Adaptation)

현재 가장 많이 사용하는 기법이다.



핵심 아이디어

Weight

W

를 직접 수정하지 않는다.

대신

$\Delta W = BA$

를 학습한다.

원래

W (4096×4096)

였다면

LoRA 는

A 4096×16

B 16×4096

만 학습한다.

즉

4096^2

↓

$4096 \times 16 \times 2$

으로 줄어든다.

Rank(r)

LoRA 의 핵심 Hyperparameter

r = 4

r = 8

r = 16

r = 32

r = 64

일반적으로

$r=8$

또는

$r=16$

를 가장 많이 사용한다.

LoRA 계산식

원래

$$Y = WX$$

LoRA

$$Y = (W + BA)X$$

여기서

W

Frozen

A

Trainable

B

Trainable

3. QLoRA

2023 년 가장 큰 혁신 중 하나이다.

논문

QLoRA:

Efficient Finetuning of Quantized LLMs

핵심은 기존

16bit

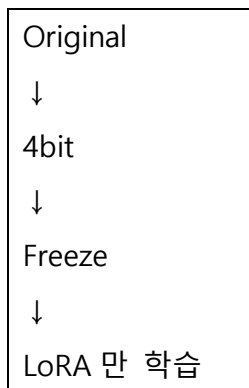
모델을

4bit

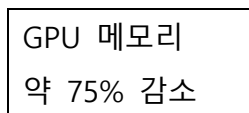
로 양자화한다.

그리고 LoRA Adapter 만 학습한다.

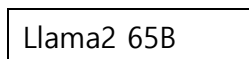
즉



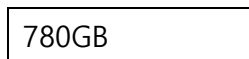
장점



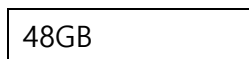
예시



기존



QLoRA

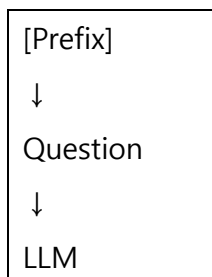


GPU 에서도 학습 가능

4. Prefix Tuning

Prompt 앞부분에 Virtual Token 을 추가한다.

예



Prefix Vector 만 학습한다. 모델은 그대로 유지한다.

5. Prompt Tuning

실제 텍스트가 아니다.

Embedding 공간의 Soft Prompt 를 학습한다.

즉 Virtual Embedding 만 학습한다.

6. P-Tuning v2

Prompt Tuning 개선판

모든 Layer 에 Prompt 를 삽입한다. 성능이 크게 향상되었다.

7. IA3

Attention Layer 의

Key

Value

FFN

Scaling 만 학습한다.

학습 파라미터가 매우 적다.

8. AdaLoRA

LoRA 의 Rank 를 자동 조절한다.

중요한 Layer Rank ↑

덜 중요한 Layer Rank ↓

자동 최적화된다.

9. DoRA

LoRA 개선 모델이다.

Weight 를 Magnitude, Direction 으로 분리한다.

LoRA 보다 성능이 좋은 경우가 많다.

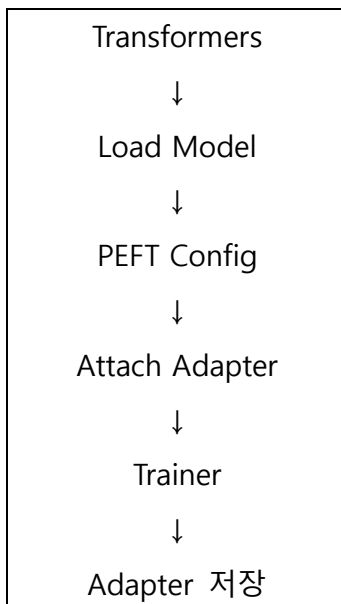
10. VeRA

Low Rank 대신 Vector 기반 Adapter 메모리를 더 절약한다.

6. PRFT 라이브러리

Hugging Face 는 **PEFT** 라이브러리를 제공하여 다양한 파라미터 효율적 파인튜닝 기법을 통합 지원한다. 대표적으로 LoRA, AdaLoRA, IA3, Prompt Tuning, Prefix Tuning 등을 동일한 인터페이스로 사용할 수 있으며, Transformers 및 TRL 과 함께 많이 사용된다.

대표적인 학습 흐름은 다음과 같다.



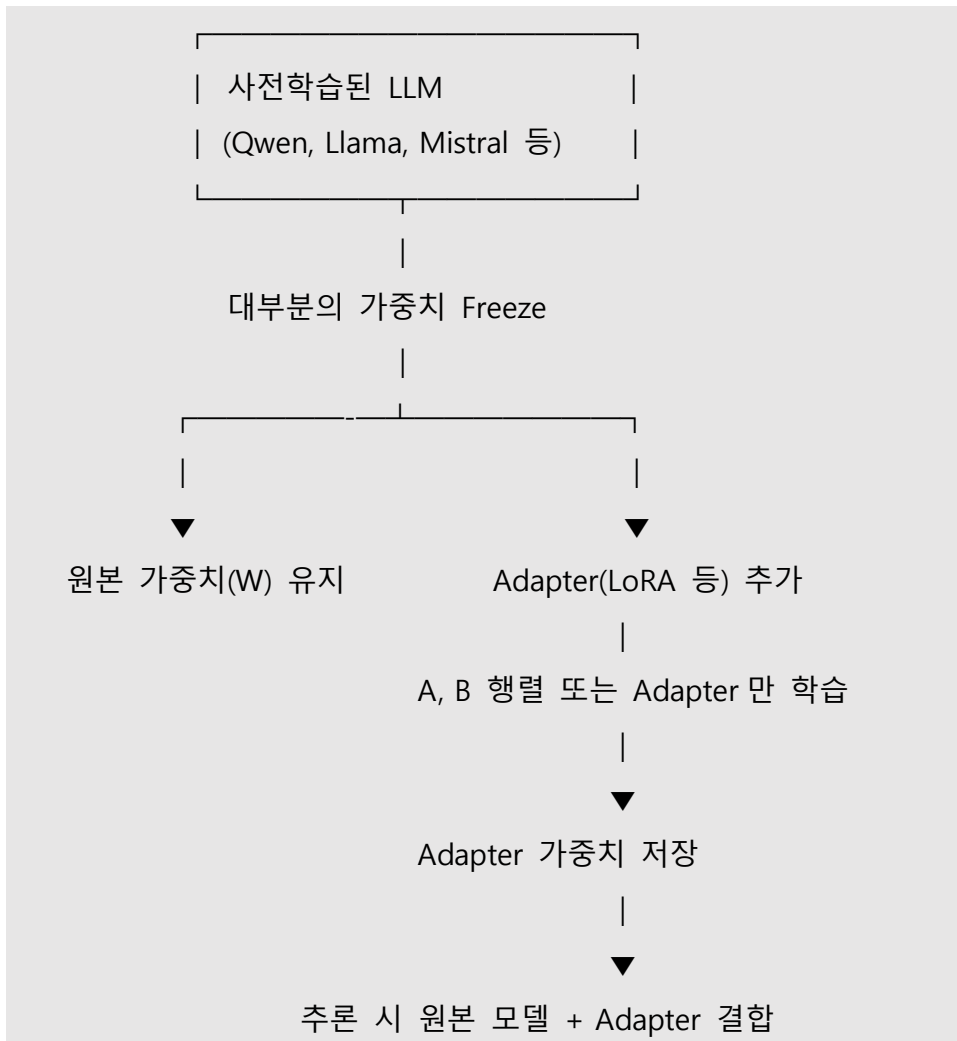
7. PRFT와 양자화의 관계

PEFT 는 양자화와 함께 사용할 때 효과가 더욱 커진다.

기법	목적
PEFT	학습 파라미터 감소
Quantization	모델 메모리 감소
LoRA	저랭크 행렬만 학습
QLoRA	4bit 양자화 + LoRA

즉, Quantization + PEFT = 최소 GPU 로 최대 모델 학습이 가능하다.

8 PRFT 학습 과정



9. 주요 PRFT 기법 비교

기법	학습 대상	메모리 효율	성능	현재 활용도
Adapter Tuning	Adapter MLP	높음	높음	중간
LoRA	저랭크 행렬(A, B)	매우 높음	매우 높음	매우 높음
QLoRA	4bit 모델 + LoRA	최고	매우 높음	매우 높음
Prompt Tuning	Soft Prompt	최고	중간	중간
Prefix Tuning	Prefix 벡터	매우 높음	중간~높음	중간

기법	학습 대상	메모리 효율	성능	현재 활용도
P-Tuning v2	Layer 별 Prompt	높음	높음	중간
IA3	Attention/FFN 스케일	매우 높음	높음	낮음~중간
AdaLoRA	적응형 Rank	매우 높음	매우 높음	높음
DoRA	방향·크기 분리	높음	매우 높음	증가 추세
VeRA	벡터 기반 Adapter	매우 높음	높음	연구·실험 단계

10. 실무에서 가장 많이 사용하는 조합

2026 년 기준으로 경량 LLM 파인튜닝에서는 다음과 같은 조합이 널리 사용된다.

목적	권장 조합
교육 및 첫 실습	Qwen2.5-1.5B + LoRA
단일 GPU 환경(24GB 내외)	Qwen3-4B + QLoRA
한국어 도메인 튜닝	Qwen 계열 + LoRA/QLoRA + SFT
대화형 챗봇	LoRA + Chat Template + SFT
기업 문서 질의응답	QLoRA + RAG(벡터 검색)
추론 성능 유지와 메모리 절감	4bit 양자화 + QLoRA + bfloat16 연산

한계와 고려사항

PEFT 는 매우 효율적이지만 다음 사항을 고려해야 한다.

- **대규모 지식 자체를 새로 학습시키는 데는 한계**가 있으며, 새로운 사실을 지속적으로 반영해야 한다면 RAG 와 병행하는 것이 일반적이다.
- **LoRA Rank(r)** 가 너무 낮으면 표현력이 부족하고, 너무 높으면 메모리 절감 효과가 감소한다.
- **Adapter 간 호환성**은 기본 모델 버전과 토큰라이저가 일치해야 보장된다.
- **도메인 특화 데이터의 품질**이 결과에 큰 영향을 미친다. 데이터 품질이 낮으면 PEFT 의 장점이 충분히 나타나지 않을 수 있다.

11. 결론

PEFT 는 거대 언어 모델을 **전체 재학습하지 않고도 적은 자원으로 새로운 작업에 적응시키는 핵심 기술**이다. 특히 LoRA 와 QLoRA 는 메모리 사용량과 학습 시간을 크게 줄이면서도 높은 성능을 유지할 수 있어 현재 가장 널리 활용된다.

실무에서는 다음과 같은 조합이 가장 일반적이다.

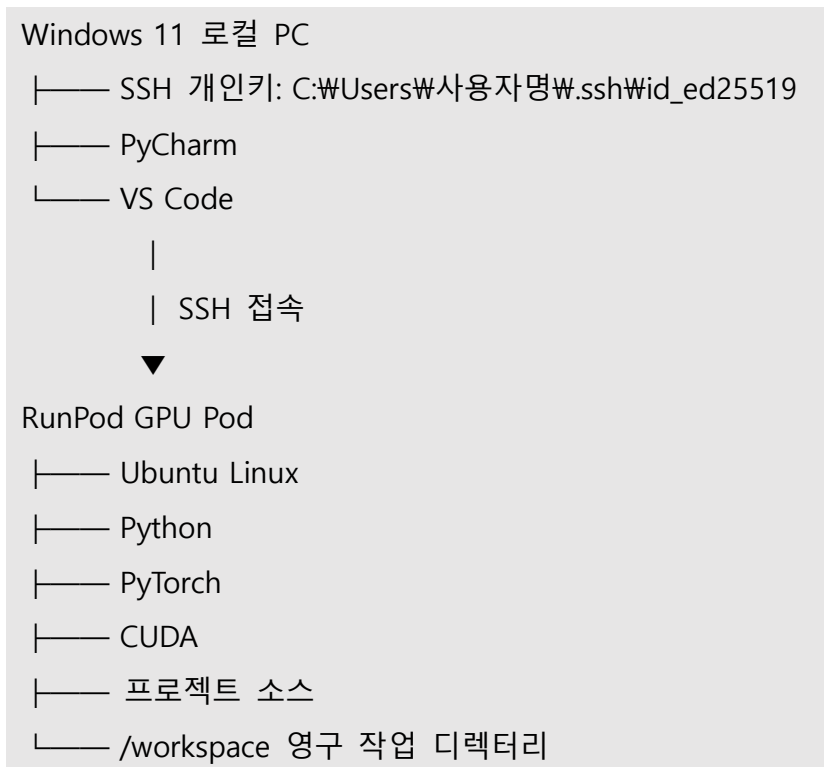
- **모델:** Qwen 계열, Llama 계열
- **튜닝 방식:** LoRA 또는 QLoRA
- **학습 방법:** 지도 미세조정(SFT)
- **지식 확장:** RAG 와 결합
- **배포:** 원본 모델 + Adapter 만 관리

이와 같은 구조는 교육용 실습부터 기업용 도메인 챗봇, 상담 시스템, 문서 질의응답 서비스까지 폭넓게 적용되고 있다.

2. 개발툴에서 RunPod SSH 접속

Windows 11 로컬 PC 에서 RunPod GPU Pod 를 생성하고, SSH 키 인증을 사용하여 PyCharm 또는 VS Code 로 원격 개발하는 절차입니다.

전체 구조는 다음과 같습니다.



RunPod 공식 PyTorch 템플릿은 일반적으로 SSH 서버, Python, CUDA, PyTorch, JupyterLab 환경이 미리 구성되어 있어 처음 사용할 때 가장 안전합니다. ([Runpod Documentation](#))

1. 사전 준비

로컬 Windows PC 에 다음 프로그램을 준비합니다.

1.1 필수 프로그램

1. Windows 10 또는 Windows 11
2. RunPod 계정
3. Windows OpenSSH Client
4. PyCharm Professional 또는 VS Code
5. 인터넷 연결

PyCharm 에서 **SSH 원격 Python 인터프리터**를 사용하려면 PyCharm Professional 이 필요합니다. Community Edition 은 SSH 원격 인터프리터와 SFTP 기반 배포 기능을 지원하지 않습니다. ([JetBrains](#))

VS Code 는 무료 버전에서도 Remote - SSH 확장 기능으로 RunPod 에 접속할 수 있습니다. ([Visual Studio Code](#))

2. Windows 에서 SSH 사용 가능 여부 확인

2.1 PowerShell 실행

Windows 시작 메뉴에서 다음 순서로 실행합니다.

시작 메뉴

→ PowerShell 검색

→ Windows PowerShell 실행

다음 명령을 입력합니다.

```
ssh -V
```

정상적인 경우 다음과 비슷하게 출력됩니다.

```
OpenSSH_for_Windows_9.x
```

이 결과가 나오면 SSH 를 바로 사용할 수 있습니다.

2.2 ssh 명령을 찾을 수 없는 경우

다음 메뉴를 엽니다.

Windows 설정

→ 시스템

→ 선택적 기능

→ 기능 보기

→ OpenSSH Client 검색

→ 설치

설치가 완료되면 PowerShell 을 닫았다가 다시 실행한 후 확인합니다.

```
ssh -V
```


3. SSH 키 생성

SSH 키는 비밀번호 대신 서버 접속을 인증하는 파일입니다.

두 개의 파일이 생성됩니다.

id_ed25519 : 개인키, 절대로 외부에 공개하면 안 됨

id_ed25519.pub : 공개키, RunPod 에 등록하는 파일

3.1 .ssh 폴더 확인

PowerShell 에서 다음 명령을 실행합니다.

```
cd $HOME
```

```
mkdir .ssh -Force
```

```
cd .ssh
```

현재 위치를 확인합니다.

```
pwd
```

예시:

```
C:\Users\사용자명\.ssh
```

3.2 SSH 키 생성 명령

다음 명령을 실행합니다.

```
ssh-keygen -t ed25519 -C "본인_RunPod_이메일"
```

예시:

```
ssh-keygen -t ed25519 -C "student@example.com"
```

다음 질문이 나타납니다.

```
Enter file in which to save the key
```

기본 경로를 사용할 것이므로 Enter 를 누릅니다.

```
C:\Users\사용자명\.ssh\id_ed25519
```

다음 질문이 나타납니다.

```
Enter passphrase
```

처음 실습할 때는 Enter 를 눌러 비워 둘 수 있습니다.

다시 확인하는 질문에서도 Enter 를 누릅니다.

정상적으로 생성되면 다음 파일이 만들어집니다.

C:\Users\사용자명\ssh\id_ed25519

C:\Users\사용자명\ssh\id_ed25519.pub

RunPod 공식 문서에서도 ED25519 방식의 SSH 키 생성을 안내하고 있습니다. ([Runpod Documentation](#))

3.3 공개키 내용 확인

PowerShell 에서 다음 명령을 실행합니다.

```
Get-Content $HOME\ssh\id_ed25519.pub
```

또는 다음 명령을 사용할 수 있습니다.

```
type $HOME\ssh\id_ed25519.pub
```

출력 예시는 다음과 같습니다.

```
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIxxxxxxxxxxxxxxxxx student@example.com
```

출력된 한 줄 전체를 복사합니다.

주의할 점:

ssh-ed25519 부터 이메일까지 한 줄 전체를 복사해야 합니다.

개인키인 다음 파일의 내용은 복사하거나 공유하면 안 됩니다.

id_ed25519

4. RunPod 계정에 SSH 공개키 등록

4.1 RunPod 로그인

RunPod 콘솔에 로그인합니다.

4.2 SSH 공개키 등록

RunPod 콘솔에서 다음 메뉴를 찾습니다.

사용자 프로필 또는 Settings

→ SSH Public Keys

→ Add SSH Key

화면 이름은 콘솔 업데이트에 따라 다음과 같이 표시될 수도 있습니다.

Settings

→ SSH Keys

이름을 입력합니다.

예: windows-main-pc

앞에서 복사한 공개키를 붙여 넣습니다.

ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAI... student@example.com

저장합니다.

중요한 점은 SSH 키를 가능하면 **Pod 를 만들기 전에 등록하는 것**입니다. 실행 중인 Pod 에 나중에 키를 추가하면 해당 Pod 의 authorized_keys 에 자동 반영되지 않을 수 있으며, 이때는 Pod 를 재배포하거나 웹 터미널에서 직접 공개키를 추가해야 합니다. ([Runpod Documentation](#))

5. RunPod 에서 새로운 Pod 만들기

5.1 Pod 생성 화면 열기

RunPod 콘솔에서 다음 중 하나를 선택합니다.

왼쪽 메뉴 Pods

→ Deploy Pod

새 UI에서는 다음 경로일 수 있습니다.

오른쪽 상단 + New → Pod

RunPod 는 새 배포 화면과 기존 배포 화면을 순차적으로 적용하고 있어 계정에 따라 메뉴 배치가 약간 다를 수 있습니다. ([Runpod Documentation](#))

5.2 템플릿 선택

처음에는 공식 템플릿을 선택하는 것이 좋습니다.

검색창에 다음과 같이 입력합니다.

RunPod PyTorch

선택 예시:

RunPod PyTorch

또는 버전이 표시된 공식 템플릿:

RunPod PyTorch 2.x

CUDA 12.x

Ubuntu 22.04

템플릿을 선택할 때 다음을 확인합니다.

Publisher 또는 Maintainer: RunPod

Official 표시

PyTorch 설치 여부

CUDA 설치 여부

SSH 지원 여부

JupyterLab 지원 여부

공식 RunPod PyTorch 템플릿은 SSH over exposed TCP 연결을 지원합니다. ([Runpod Documentation](#))

5.3 Pod 이름 입력

식별하기 쉬운 이름을 입력합니다.

예: pytorch-study-pod

또는:

fastapi-rag-pod

Pod 이름은 SSH 연결 주소가 아니라 RunPod 화면에서 Pod 를 구분하기 위한 이름입니다.

5.4 SSH Terminal Access 활성화

다음 옵션을 반드시 활성화합니다.

SSH Terminal Access

화면에 공개키 입력란이 나타나면 앞에서 생성한 공개키를 넣습니다.

ssh-ed25519 AAAAC3... 본인이메일

계정에 공개키를 이미 등록했다면 자동으로 선택되거나 반영될 수 있습니다.

VS Code 와 PyCharm 에서 직접 SSH 로 접속하려면 Pod 가 SSH over exposed TCP 를 지원해야 합니다. ([Runpod Documentation](#))

5.5 Jupyter Notebook 옵션

처음에는 다음 옵션도 활성화하는 것이 좋습니다.

Start Jupyter Notebook

SSH 연결에 문제가 생겼을 때 RunPod 웹 화면의 JupyterLab 이나 웹 터미널을 이용해 Pod 내부를 확인할 수 있습니다.

5.6 GPU 선택

학습 목적에 따라 GPU 를 선택합니다.

기본 실습이나 소형 모델:

RTX 3090

RTX 4090

A4000

A5000

중대형 모델:

A6000

A100

H100

처음 실습에는 비용과 성능을 고려해 다음이 적합합니다.

RTX 3090 24GB

RTX 4090 24GB

확인할 항목:

GPU VRAM

시간당 비용

현재 사용 가능 여부

Secure Cloud 또는 Community Cloud

Secure Cloud 와 Community Cloud

Secure Cloud

- 비교적 안정적인 환경
- 기업 또는 중요 프로젝트에 적합
- 일반적으로 비용이 더 높음

Community Cloud

- 상대적으로 저렴함
- 호스트와 가용성이 다양할 수 있음
- 교육 및 실습에 적합

6. 디스크와 볼륨 설정

RunPod 에서 디스크는 보통 두 종류로 구분됩니다.

6.1 Container Disk

컨테이너 운영체제와 설치 패키지가 저장됩니다.

예시:

Container Disk: 20GB~50GB

PyTorch 패키지와 모델을 추가로 설치한다면 다음 정도가 편리합니다.

40GB 이상

6.2 Volume Disk

프로젝트 코드와 모델 파일을 보관하는 작업 공간입니다.

일반적으로 다음 경로에 연결됩니다.

/workspace

예시:

Volume Disk: 50GB

Hugging Face 모델이나 학습 데이터를 많이 내려받는다면 다음처럼 늘립니다.

100GB

200GB

RunPod 공식 문서에서는 /workspace 를 기본 영구 저장 작업 디렉터리로 안내합니다. 반면 /tmp 는 임시 디렉터리이므로 Pod 중지나 재배포 과정에서 데이터가 유지되지 않을 수 있습니다. ([Runpod Documentation](#))

6.3 권장 설정 예시

처음 실습용 설정:

Template: RunPod PyTorch

GPU: RTX 3090 또는 RTX 4090

Container Disk: 40GB

Volume Disk: 50GB

SSH Terminal Access: 활성화

Start Jupyter Notebook: 활성화

Volume Mount Path: /workspace

7. Pod 배포

화면 아래쪽에서 가격을 확인합니다.

GPU 시간당 비용

Volume 비용

Container Disk 비용

총 예상 비용

그다음 다음 버튼을 누릅니다.

Deploy

Pod 상태가 다음 순서로 바뀝니다.

Creating

→ Starting

→ Running

Running 상태가 될 때까지 기다린 후 연결합니다.

8. RunPod SSH 연결 정보 확인

실행 중인 Pod 에서 다음을 선택합니다.

Pods

→ 생성한 Pod 선택

→ Connect

→ SSH

여러 연결 명령이 표시될 수 있습니다.

그중 다음 제목의 명령을 찾습니다.

SSH over exposed TCP

예시:

```
ssh root@213.173.108.12 -p 17445 -i ~/.ssh/id_ed25519
```

각 항목의 의미는 다음과 같습니다.

Root → RunPod 접속 사용자 이름

213.173.108.12 → Pod의 공인 IP 주소

17445 → 외부에서 사용하는 SSH 포트 번호

~/.ssh/id_ed25519 → 로컬 PC의 SSH 개인키 위치

RunPod는 Pod의 Connect 탭에서 실제 IP 주소와 포트가 포함된 SSH 명령을 제공합니다.

[\(Runpod Documentation\)](#)

9. Windows PowerShell에서 SSH 접속 테스트

IDE에 연결하기 전에 PowerShell에서 먼저 접속을 확인해야 합니다.

RunPod에서 복사한 명령이 다음과 같다고 가정합니다.

```
ssh root@213.173.108.12 -p 17445 -i ~/.ssh/id_ed25519
```

Windows PowerShell에서는 다음과 같이 실행할 수 있습니다.

```
ssh root@213.173.108.12 -p 17445 -i $HOME\ssh\id_ed25519
```

또는 전체 경로를 사용합니다.

```
ssh root@213.173.108.12 -p 17445 -i C:\Users\사용자명\ssh\id_ed25519
```

처음 접속하면 다음 질문이 나타날 수 있습니다.

Are you sure you want to continue connecting (yes/no/[fingerprint])?

다음을 입력합니다.

yes

정상적으로 접속되면 프롬프트가 다음과 비슷하게 바뀝니다.

root@xxxxxxx:/#

10. RunPod 내부 환경 확인

SSH 접속 후 다음 명령을 차례대로 실행합니다.

10.1 현재 사용자 확인

whoami

정상 결과:

root

10.2 현재 위치 확인

pwd

10.3 작업 디렉터리 이동

cd /workspace

pwd

정상 결과:

/workspace

10.4 GPU 확인

nvidia-smi

정상적으로 GPU 이름, VRAM, CUDA 관련 정보가 표시되어야 합니다.

10.5 Python 확인

`python --version`

또는:

`python3 --version`

10.6 PyTorch GPU 확인

```
python -c "import torch; print('torch:', torch.__version__); print('cuda:', torch.cuda.is_available());  
print('gpu:', torch.cuda.get_device_name(0) if torch.cuda.is_available() else '없음')"
```

정상 예시:

torch: 2.x.x

cuda: True

gpu: NVIDIA GeForce RTX 4090

10.7 SSH 종료

`exit`

11. SSH 설정 파일 만들기

매번 긴 SSH 명령을 입력하지 않으려면 Windows SSH 설정 파일을 작성합니다.

파일 위치:

`C:\Users\사용자명\.ssh\config`

PowerShell 에서 다음 명령을 실행합니다.

`notepad $HOME\.ssh\config`

파일이 없다는 메시지가 나오면 새로 생성합니다.

다음 내용을 입력합니다.

Host runpod-gpu

HostName 213.173.108.12

User root

Port 17445

IdentityFile C:/Users/사용자명/.ssh/id_ed25519

ServerAliveInterval 30

ServerAliveCountMax 120

실제 값으로 변경해야 하는 부분:

HostName → RunPod 에서 제공한 IP

Port → RunPod 에서 제공한 SSH 포트

IdentityFile → 본인의 개인키 경로

예시:

Host runpod-fastapi

HostName 123.45.67.89

User root

Port 24567

IdentityFile C:/Users/yoony/.ssh/id_ed25519

ServerAliveInterval 30

ServerAliveCountMax 120

저장 후 PowerShell 에서 다음처럼 접속할 수 있습니다.

ssh runpod-fastapi

12. PyCharm 에서 RunPod 사용 방법

PyCharm 에서는 두 가지 방법을 사용할 수 있습니다.

방법 1. 로컬 프로젝트 + RunPod 원격 인터프리터

방법 2. RunPod 프로젝트를 PyCharm Remote Development 로 직접 열기

처음에는 **방법 1** 이 이해하기 쉽습니다.

13. PyCharm 방법 1: SSH 원격 Python 인터프리터 설정

13.1 지원 버전 확인

다음 기능은 PyCharm Professional 에서 사용할 수 있습니다.

SSH Interpreter

SFTP Deployment

Remote Development

PyCharm Community Edition 만 있다면 VS Code Remote-SSH 사용을 권장합니다. PyCharm 의 SSH 인터프리터 기능은 Pro 전용 플러그인에 의존합니다. ([JetBrains](https://www.jetbrains.com/pycharm/features/remote-development.html))

13.2 로컬 프로젝트 만들기

PyCharm 에서 다음과 같이 프로젝트를 만듭니다.

File → New Project

예시 경로: C:\WLLM_workspace\wrunpod_test_project

프로젝트 안에 main.py 를 만듭니다.

```
# PyTorch 라이브러리를 불러옵니다.
import torch

# 현재 실행 중인 PyTorch 버전을 출력합니다.
print("PyTorch 버전:", torch.__version__)

# CUDA GPU 를 사용할 수 있는지 확인합니다.
print("CUDA 사용 가능:", torch.cuda.is_available())

# CUDA 를 사용할 수 있다면 현재 GPU 이름을 출력합니다.
if torch.cuda.is_available():
    print("GPU 이름:", torch.cuda.get_device_name(0))
else:
    print("사용 가능한 CUDA GPU 가 없습니다.")
```

아직 실행하지 않아도 됩니다.

13.3 SSH 설정 등록

PyCharm 메뉴에서 다음으로 이동합니다.

File

→ Settings

→ Tools

→ SSH Configurations

단축키:

Ctrl + Alt + S

왼쪽 위의 + 버튼을 누릅니다.

다음과 같이 입력합니다.

Host: RunPod 에서 제공한 IP

Port: RunPod 에서 제공한 포트

Username: root

Authentication type: Key pair

Private key file: C:\Users\사용자명\.ssh\id_ed25519

Passphrase: 키 생성 시 설정했을 경우만 입력

예시:

Host: 123.45.67.89

Port: 24567

Username: root

Private key: C:\Users\Wyoona\.ssh\id_ed25519

Test Connection 또는 Check Connection 을 누릅니다.

정상 결과:

Successfully connected

PyCharm 은 Tools → SSH Configurations 에서 저장한 연결을 원격 인터프리터, SFTP 배포 및 SSH 터미널에 재사용할 수 있습니다. ([JetBrains](#))

13.4 SSH 원격 인터프리터 추가

다음 메뉴로 이동합니다.

File

→ Settings

→ Project: 프로젝트명

→ Python Interpreter

다음 순서로 선택합니다.

Add Interpreter

→ On SSH

PyCharm 버전에 따라 다음과 같이 표시될 수도 있습니다.

Add New Interpreter

→ On SSH

앞에서 만든 SSH 설정을 선택합니다.

Existing

→ RunPod SSH 설정 선택

또는 직접 입력합니다.

Host: RunPod IP

Port: RunPod SSH 포트

Username: root

Authentication: Private key

PyCharm 공식 절차도 Python Interpreter → Add Interpreter → On SSH 순서입니다. ([JetBrains](#))

13.5 원격 Python 경로 선택

PyCharm 이 SSH 접속에 성공하면 원격 Python 실행 파일을 선택합니다.

일반적으로 다음 중 하나입니다.

/usr/bin/python

/usr/bin/python3

/usr/local/bin/python

/usr/local/bin/python3

RunPod 터미널에서 먼저 확인하면 정확합니다.

which python

또는:

which python3

예시 결과:

/usr/bin/python

이 경로를 PyCharm 의 Interpreter 경로에 입력합니다.

13.6 프로젝트 동기화 경로 설정

로컬 프로젝트와 RunPod 의 원격 디렉토리를 연결합니다.

권장 원격 경로:

/workspace/runpod_test_project

예시:

Local path:

C:\WLLM_workspace\runpod_test_project

Remote path:

/workspace/runpod_test_project

가능하면 /root 가 아니라 /workspace 아래에 프로젝트를 둡니다.

13.7 자동 업로드 설정

다음 메뉴로 이동합니다.

File

→ Settings

→ Build, Execution, Deployment

→ Deployment

SFTP 설정을 확인합니다.

Mappings 탭에서 다음을 지정합니다.

Local path:

C:\WLLM_workspace\runpod_test_project

Deployment path:

/workspace/runpod_test_project

자동 업로드를 설정하려면:

Tools

→ Deployment

→ Options

→ Upload changed files automatically to the default server

다음 값 중 하나를 선택합니다.

Always

이제 로컬에서 파일을 저장하면 RunPod 로 업로드됩니다.

13.8 PyCharm 에서 실행 확인

우측 하단 또는 상단의 Python 인터프리터가 RunPod SSH 인터프리터인지 확인합니다.

표시 예시:

Python 3.x (SSH: root@RunPod)

main.py 에서 마우스 오른쪽 버튼을 누릅니다.

Run 'main'

정상 결과 예시:

PyTorch 버전: 2.x.x

CUDA 사용 가능: True

GPU 이름: NVIDIA GeForce RTX 4090

이 결과가 나오면 코드는 로컬 컴퓨터가 아니라 RunPod GPU 환경에서 실행된 것입니다.

14. PyCharm 에서 RunPod SSH 터미널 열기

다음 메뉴를 선택합니다.

Tools

→ Start SSH Session

→ 등록된 RunPod 연결

버전에 따라 다음과 같이 표시될 수 있습니다.

Tools

→ SSH Session

SSH 터미널에서 확인합니다.

```
cd /workspace
```

```
nvidia-smi
```

PyCharm 은 등록된 SSH 연결 또는 기본 원격 인터프리터를 이용해 전용 SSH 터미널을 열 수 있습니다. ([JetBrains](#))

15. PyCharm 방법 2: Remote Development 사용

이 방식은 프로젝트 전체와 PyCharm 실행 백엔드를 RunPod 에서 실행합니다.

즉:

로컬 PC

→ 화면과 키보드 입력 담당

RunPod

→ 프로젝트 파일, Python, 인덱싱, 실행, 디버깅 담당

15.1 시작 화면에서 연결

PyCharm 을 실행한 후 Welcome 화면에서 다음을 선택합니다.

Remote Development

→ SSH

→ New Connection

PyCharm 이 이미 열려 있다면:

File

→ Remote Development

다음 정보를 입력합니다.

Host: RunPod IP

Port: RunPod SSH 포트

User: root

Authentication: Key pair

Private key: C:\Users\사용자명\.ssh\id_ed25519

Check Connection and Continue 을 누릅니다.

PyCharm 공식 Remote Development 절차도 Welcome 화면 또는 File → Remote Development 에서 SSH 연결을 만든 후 원격 IDE 백엔드를 시작하는 방식입니다. ([JetBrains](#))

15.2 원격 IDE 설치 위치

가능하면 설치 경로를 /workspace 아래에 지정합니다.

예시:

/workspace/.cache/JetBrains

기본 /root 경로에 설치하면 Pod 재배포나 컨테이너 변경 시 사라질 가능성이 더 큽니다.

15.3 원격 프로젝트 경로 선택

예시:

/workspace/runpod_test_project

프로젝트 폴더가 없다면 웹 터미널이나 SSH 에서 생성합니다.

mkdir -p /workspace/runpod_test_project

그다음 다음 버튼을 누릅니다.

Start IDE and Connect

PyCharm 이 RunPod 에 백엔드 IDE 를 설치한 후 JetBrains Client 창을 엽니다. ([JetBrains](#))

16. VS Code 에서 RunPod SSH 접속

VS Code 방식은 무료이고 설정이 비교적 간단합니다.

16.1 VS Code 확장 프로그램 설치

VS Code 를 실행합니다.

왼쪽 메뉴에서 Extensions 를 선택합니다.

단축키:

Ctrl + Shift + X

검색창에 다음을 입력합니다.

Remote - SSH

다음 확장을 설치합니다.

Remote - SSH

Publisher: Microsoft

Python 개발을 위해 다음 확장도 설치합니다.

Python

Publisher: Microsoft

RunPod 와 VS Code 공식 문서 모두 원격 SSH 개발을 위해 Remote - SSH 확장 설치를 안내합니다. ([Runpod Documentation](#))

16.2 SSH Host 추가

RunPod Pod 화면에서 다음으로 이동합니다.

Pod

→ Connect

→ SSH

→ SSH over exposed TCP

다음 형태의 명령을 복사합니다.

```
ssh root@123.45.67.89 -p 24567 -i ~/.ssh/id_ed25519
```

VS Code 에서 명령 팔레트를 엽니다. Ctrl + Shift + P

다음을 검색합니다.

Remote-SSH: Add New SSH Host

복사한 SSH 명령을 붙여 넣습니다.

```
ssh root@123.45.67.89 -p 24567 -i C:\Users\사용자명\.ssh\id_ed25519
```

SSH 설정 파일을 선택하라는 메시지가 나오면 다음 파일을 선택합니다.

C:\Users\사용자명\.ssh\config

RunPod 공식 가이드도 Remote-SSH: Add New SSH Host 에 RunPod 가 제공한 SSH 명령을 등록하도록 안내합니다. ([Runpod Documentation](#))

16.3 SSH config 직접 작성

VS Code 에서 명령 팔레트를 엽니다.

Ctrl + Shift + P

다음을 선택합니다.

Remote-SSH: Open SSH Configuration File

Windows 사용자 SSH 설정 파일을 선택합니다.

C:\Users\사용자명\.ssh\config

다음 내용을 추가합니다.

Host runpod-gpu

HostName 123.45.67.89

User root

Port 24567

IdentityFile C:/Users/사용자명/.ssh/id_ed25519

ServerAliveInterval 30

ServerAliveCountMax 120

예시:

Host runpod-rag

HostName 123.45.67.89

User root

Port 24567

IdentityFile C:/Users/yoona/.ssh/id_ed25519

ServerAliveInterval 30

ServerAliveCountMax 120

저장합니다.

16.4 RunPod 접속

명령 팔레트를 엽니다.

Ctrl + Shift + P

다음을 선택합니다.

Remote-SSH: Connect to Host

등록한 호스트를 선택합니다.

runpod-gpu

처음 연결할 때 운영체제를 묻는다면 다음을 선택합니다.

Linux

새 VS Code 창이 열립니다.

왼쪽 아래 상태 표시줄에 다음처럼 표시되면 정상입니다.

SSH: runpod-gpu

VS Code 는 접속 과정에서 원격 Pod 에 VS Code Server 를 자동 설치하고, 원격 파일 편집·코드 완성·실행·디버깅 기능을 제공합니다. ([Visual Studio Code](#))

17. VS Code 에서 원격 프로젝트 열기

연결된 VS Code 창에서 다음을 선택합니다.

File

→ Open Folder

다음 경로를 입력합니다.

/workspace

또는 프로젝트 경로를 입력합니다.

/workspace/runpod_test_project

폴더가 없다면 터미널에서 생성합니다.

```
mkdir -p /workspace/runpod_test_project
```

RunPod 공식 가이드도 원격 연결 후 일반적으로 /workspace 폴더를 열도록 안내합니다.
([Runpod Documentation](#))

18. VS Code 원격 터미널 사용

VS Code 메뉴에서 다음을 선택합니다.

Terminal

→ New Terminal

이 터미널은 Windows PowerShell 이 아니라 RunPod 의 Ubuntu 터미널입니다.

확인:

```
whoami
```

결과:

```
root
```

현재 위치 확인:

```
pwd
```

GPU 확인:

```
nvidia-smi
```

Python 확인:

```
python --version
```

19. VS Code 원격 Python 인터프리터 선택

명령 팔레트를 엽니다.

Ctrl + Shift + P

다음을 선택합니다.

Python: Select Interpreter

RunPod 내부의 Python 을 선택합니다.

예시:

/usr/bin/python

/usr/local/bin/python

목록에 없다면 다음을 선택합니다.

Enter interpreter path

→ Find

터미널에서 경로를 확인할 수도 있습니다.

which python

예시 결과:

/usr/bin/python

20. VS Code 에서 GPU 코드 실행

main.py 파일을 만듭니다.

```
# PyTorch 라이브러리를 불러옵니다.
import torch

# 현재 PyTorch 버전을 출력합니다.
print("PyTorch 버전:", torch.__version__)

# CUDA 사용 가능 여부를 확인합니다.
cuda_available = torch.cuda.is_available()
print("CUDA 사용 가능:", cuda_available)

# CUDA 가 사용 가능하면 GPU 이름과 메모리 정보를 출력합니다.
if cuda_available:
    # 첫 번째 GPU 의 장치 이름을 가져옵니다.
    gpu_name = torch.cuda.get_device_name(0)

    # 첫 번째 GPU 의 전체 메모리 크기를 byte 단위로 가져옵니다.
    total_memory = torch.cuda.get_device_properties(0).total_memory
```

```
# byte 단위 값을 GB 단위로 변환합니다.
total_memory_gb = total_memory / (1024 ** 3)

# 확인 결과를 출력합니다.
print("GPU 이름:", gpu_name)
print(f"GPU 전체 메모리: {total_memory_gb:.2f} GB")
else:
    print("CUDA GPU 를 사용할 수 없습니다.")
```

우측 상단 실행 버튼을 누르거나 터미널에서 실행합니다.

```
python main.py
```

정상 결과 예시:

PyTorch 버전: 2.x.x

CUDA 사용 가능: True

GPU 이름: NVIDIA GeForce RTX 4090

GPU 전체 메모리: 23.65 GB

21. 프로젝트 가상환경 만들기

공식 PyTorch 이미지에 패키지가 이미 설치되어 있지만, 프로젝트별 환경을 분리하려면 가상환경을 사용합니다.

21.1 프로젝트 폴더 생성

```
cd /workspace
mkdir -p runpod_test_project
cd runpod_test_project
```

21.2 가상환경 생성

```
python -m venv .venv
```

venv 모듈이 없다는 오류가 나면 다음을 실행합니다.

```
apt update
```

```
apt install -y python3-venv
```


다시 생성합니다.

```
python -m venv .venv
```

21.3 가상환경 활성화

```
source .venv/bin/activate
```

프롬프트 앞에 다음이 표시됩니다.

(.venv)

21.4 pip 업그레이드

```
python -m pip install --upgrade pip setuptools wheel
```

21.5 패키지 설치

FastAPI 예시:

```
pip install fastapi uvicorn python-dotenv
```

설치 패키지 저장:

```
pip freeze > requirements.txt
```

22. VS Code 에서 가상환경 선택

명령 팔레트를 엽니다.

Ctrl + Shift + P

다음을 선택합니다.

Python: Select Interpreter

다음 경로를 선택합니다.

```
/workspace/runpod_test_project/.venv/bin/python
```

목록에 없으면 직접 입력합니다.

Enter interpreter path

→ /workspace/runpod_test_project/.venv/bin/python

23. PyCharm 에서 가상환경 선택

PyCharm SSH 인터프리터 설정 시 원격 Python 경로를 다음으로 지정합니다.

/workspace/runpod_test_project/.venv/bin/python

이미 SSH 인터프리터를 만든 경우:

File

→ Settings

→ Python Interpreter

→ Add Interpreter

→ On SSH

기존 SSH 연결을 선택하고 원격 Python 경로로 가상환경 Python 을 지정합니다.

24. 로컬 프로젝트를 RunPod 로 복사하는 방법

24.1 SCP 사용

PowerShell 에서 실행합니다.

파일 하나 복사:

```
scp -P 24567 -i $HOME\sshWid_ed25519 .\main.py  
root@123.45.67.89:/workspace/runpod_test_project/
```

폴더 전체 복사:

```
scp -r -P 24567 -i $HOME\sshWid_ed25519 C:\WLLM_workspace\my_project  
root@123.45.67.89:/workspace/
```

주의:

ssh 명령의 포트 옵션은 -p

scp 명령의 포트 옵션은 -P

즉, SCP 에서는 대문자 P 를 사용합니다.

24.2 GitHub 사용

RunPod 터미널에서 다음을 실행합니다.

```
cd /workspace
```

```
git clone 저장소주소
```

예시:

```
git clone https://github.com/example/runpod-project.git
```

프로젝트로 이동합니다.

```
cd runpod-project
```

패키지를 설치합니다.

```
pip install -r requirements.txt
```

25. FastAPI 를 RunPod 에서 실행하고 로컬에서 접속

FastAPI 프로젝트를 다음처럼 실행한다고 가정합니다.

```
uvicorn app.main:app --host 0.0.0.0 --port 8000
```

--host 0.0.0.0 을 지정해야 외부 접속 또는 포트 포워딩이 가능합니다.

25.1 VS Code 포트 포워딩

VS Code 원격 연결 상태에서:

Ports 패널

→ Forward a Port

→ 8000 입력

또는 명령 팔레트에서:

Ports: Focus on Ports View

포트 8000 을 등록합니다.

로컬 브라우저에서 접속합니다.

```
http://localhost:8000
```

Swagger:

```
http://localhost:8000/docs
```

VS Code Remote-SSH 는 원격 포트를 로컬 컴퓨터로 전달하는 포트 포워딩 기능을 지원합니다.
([Visual Studio Code](#))

25.2 SSH 명령으로 포트 포워딩

PowerShell 에서 다음을 실행합니다.

```
ssh -L 8000:localhost:8000 root@123.45.67.89 -p 24567 -i $HOME\sshWid_ed25519
```

의미:

로컬 PC 의 8000 포트

→ SSH 터널

→ RunPod 의 localhost:8000

그다음 브라우저에서 접속합니다.

<http://localhost:8000>

26. RunPod HTTP 포트로 FastAPI 공개

Pod 생성 또는 템플릿 설정에서 HTTP 포트에 다음을 추가합니다.

8000

또는 형식에 따라:

8000/http

RunPod 는 보통 다음과 비슷한 프록시 URL 을 제공합니다.

<https://PodID-8000.proxy.runpod.net>

FastAPI 실행:

```
uvicorn app.main:app --host 0.0.0.0 --port 8000
```

Swagger 접속:

<https://PodID-8000.proxy.runpod.net/docs>

중요한 API 키나 관리자 기능이 있는 서비스를 인증 없이 외부에 공개하지 않는 것이 좋습니다.

27. 자주 발생하는 SSH 오류

27.1 Permission denied (publickey)

오류:

Permission denied (publickey)

확인 사항:

1. RunPod 에 공개키가 등록되었는가?
2. 로컬에서 올바른 개인키를 사용했는가?
3. 공개키 등록 후 Pod 를 생성했는가?
4. SSH 명령의 IP 와 포트가 현재 Pod 정보와 같은가?
5. 개인키와 공개키가 한 쌍인가?

공개키 다시 확인:

Get-Content \$HOME\ssh\id_rsa.pub

RunPod 설정의 SSH Public Key 와 비교합니다.

실행 중인 Pod 에 키가 반영되지 않았다면 RunPod 웹 터미널에서 다음처럼 추가할 수 있습니다.

```
mkdir -p ~/.ssh
```

```
chmod 700 ~/.ssh
```

```
echo "ssh-ed25519 AAAA... 본인이메일" >> ~/.ssh/authorized_keys
```

```
chmod 600 ~/.ssh/authorized_keys
```

RunPod 는 접속 시 비밀번호를 요구하지 않고 SSH 키 인증을 사용하므로, 암호 입력 화면이 나온다면 키 설정이 잘못되었을 가능성이 높습니다. ([Runpod Documentation](#))

27.2 Connection timed out

오류:

ssh: connect to host ... port ...: Connection timed out

확인 사항:

Pod 상태가 Running 인가?

Pod 초기화가 완료되었는가?

SSH Terminal Access 가 활성화되었는가?

현재 Connect 화면의 최신 IP 와 포트를 사용했는가?
회사나 학교 네트워크가 외부 SSH 포트를 차단하는가?

다른 네트워크 또는 휴대전화 테더링으로 테스트해 볼 수 있습니다.

27.3 Connection refused

오류:

Connection refused

원인:

SSH 서버가 실행되지 않음

TCP 22 번 포트가 노출되지 않음

Pod 가 아직 초기화 중

사용자 정의 템플릿의 SSH 설정 누락

공식 RunPod PyTorch 템플릿을 사용하면 이 문제를 줄일 수 있습니다. 사용자 정의 템플릿은 SSH daemon 실행과 TCP 22 포트 노출을 직접 구성해야 합니다. ([Runpod Documentation](#))

27.4 REMOTE HOST IDENTIFICATION HAS CHANGED

Pod IP 가 재사용되거나 호스트 키가 변경되면 나타날 수 있습니다.

WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED!

다음 명령으로 기존 기록을 삭제합니다.

```
ssh-keygen -R "[123.45.67.89]:24567"
```

그다음 다시 접속합니다.

```
ssh runpod-gpu
```

새 지문 확인에서 yes 를 입력합니다.

27.5 개인키 파일을 찾지 못하는 오류

오류:

Identity file ... not accessible

파일 존재 여부를 확인합니다.

```
Test-Path $HOME\$.ssh\id_ed25519
```

정상 결과:

True

파일 목록 확인:

Get-ChildItem \$HOME\ssh

27.6 VS Code 연결이 계속 실패하는 경우

VS Code 가 RunPod 에 설치한 원격 서버 파일이 손상되었을 수 있습니다.

RunPod 웹 터미널이나 일반 SSH 로 접속한 후 다음을 실행합니다.

```
rm -rf ~/.vscode-server
```

그다음 VS Code 에서 다시 연결합니다.

RunPod 공식 문제 해결 절차도 VS Code Server 설치 실패 시 .vscode-server 삭제 후 재연결하는 방법을 안내합니다. ([Runpod Documentation](#))

27.7 Pod 재시작 후 연결되지 않는 경우

Pod 를 중지한 뒤 다시 시작하면 SSH 포트가 바뀔 수 있습니다.

RunPod 에서 다시 확인합니다.

Pods

→ Pod

→ Connect

→ SSH over exposed TCP

새 IP 또는 포트를 SSH 설정 파일에 반영합니다.

Host runpod-gpu

HostName 새로운_IP

Port 새로운_포트

RunPod 공식 문서에서도 Pod 중지 후 재개하면 포트가 변경될 수 있으므로 SSH 설정을 갱신해야 한다고 안내합니다. ([Runpod Documentation](#))

28. 저장 공간 사용 시 주의사항

권장 저장 위치:

/workspace

주의할 위치:

/tmp

/root

파일 예시:

/workspace/projects

/workspace/models

/workspace/datasets

/workspace/checkpoints

권장 구조:

/workspace/

├── projects/

| └── fastapi_rag_project/

├── models/

├── datasets/

└── checkpoints/

Pod 를 완전히 삭제하면 연결된 저장소 설정에 따라 데이터도 삭제될 수 있습니다. 중요한 프로젝트는 GitHub, Google Drive, S3 또는 로컬 PC 에 별도로 백업해야 합니다.

29. 비용을 줄이는 사용 습관

GPU 작업이 끝나면 RunPod 콘솔에서 다음 중 하나를 수행합니다.

잠시 사용하지 않을 때

Stop Pod

일반적으로 GPU 실행 비용은 멈추지만 저장소 비용은 계속 발생할 수 있습니다.

더 이상 사용하지 않을 때

Terminate Pod

Terminate 전에 /workspace 데이터가 보존되는 구조인지 반드시 확인합니다.

권장 습관

1. 학습 체크포인트를 /workspace 에 저장
2. 중요한 코드를 GitHub 에 push
3. 결과 파일을 로컬로 다운로드
4. nvidia-smi 로 불필요한 프로세스 확인
5. 작업 종료 후 Pod Stop 또는 Terminate
6. RunPod 결제 화면에서 잔액과 사용량 확인

30. 최종 실행 점검표

RunPod 설정

- [] SSH 공개키를 RunPod 계정에 등록함
- [] 공식 RunPod PyTorch 템플릿을 선택함
- [] SSH Terminal Access 를 활성화함
- [] GPU 와 시간당 비용을 확인함
- [] /workspace 볼륨 크기를 확인함
- [] Pod 상태가 Running 임

SSH 확인

- [] PowerShell 에서 ssh -V 가 실행됨
- [] id_ed25519 과 id_ed25519.pub 가 존재함
- [] PowerShell SSH 접속에 성공함
- [] nvidia-smi 가 정상 출력됨
- [] torch.cuda.is_available()이 True 임

PyCharm 확인

- [] PyCharm Professional 을 사용함
- [] SSH Configuration 테스트에 성공함
- [] On SSH 인터프리터를 추가함
- [] 원격 Python 경로를 올바르게 선택함
- [] 프로젝트 경로를 /workspace 아래로 지정함
- [] 실행 결과에서 GPU 가 확인됨

VS Code 확인

- [] Remote - SSH 확장을 설치함
- [] SSH config 에 RunPod 정보를 등록함
- [] 왼쪽 아래에 SSH: 호스트명이 표시됨
- [] /workspace 프로젝트 폴더를 열었음
- [] 원격 Python 인터프리터를 선택함
- [] 원격 터미널에서 nvidia-smi 가 실행됨

권장 선택

처음 RunPod 를 사용하는 경우에는 다음 구성이 가장 단순합니다.

RunPod 공식 PyTorch 템플릿

- + SSH Terminal Access
- + /workspace 볼륨
- + VS Code Remote - SSH

PyCharm Professional 을 사용하고 기존 PyCharm 프로젝트를 유지하려면 다음 구성이 적합합니다.

로컬 PyCharm 프로젝트

- + RunPod SSH 원격 인터프리터
- + SFTP 자동 업로드

프로젝트 파일과 개발 환경을 전부 RunPod 에서 운영하려면 다음 구성이 적합합니다.

PyCharm Remote Development

- + JetBrains Gateway + /workspace 원격 프로젝트

3. Supervised Fine-tuning 실습

Supervised Fine-Tuning 전체 실습 파이프라인

이 실습은 **Qwen2.5-0.5B-Instruct** 모델을 한국어 상담 데이터로 SFT하는 예제입니다.
전체 흐름은 다음과 같습니다.

```
GPU 확인
↓
라이브러리 설치
↓
학습 데이터 생성 및 검증
↓
Tokenizer 로드
↓
사전 학습 모델 4bit 로드
↓
LoRA Adapter 설정
↓
SFTTrainer 구성
↓
모델 학습
↓
Adapter 저장
↓
학습 전·후 추론 비교
↓
저장된 Adapter 재로드
```

TRL의 SFTTrainer는 일반 텍스트뿐 아니라 messages 형태의 대화형 데이터셋을 지원하며, 대화형 데이터에는 모델의 Chat Template을 자동으로 적용할 수 있습니다. LoRA는 전체 모델을 학습하지 않고 일부 저차원 행렬만 학습하므로 GPU 메모리와 저장 공간을 줄이는 데 적합합니다. ([Hugging Face](#))

1. 프로젝트 구조

```
qwen_sft_project/  
|  
|—— data/  
|   |—— train.jsonl  
|   |—— valid.jsonl  
|  
|—— outputs/  
|   |—— checkpoints/  
|   |—— qwen2.5-korean-sft-lora/  
|  
|—— 01_create_dataset.py  
|—— 02_train_sft.py  
|—— 03_inference.py  
|—— requirements.txt  
|—— .env.example  
|—— README.md
```

RunPod Notebook 또는 Google Colab에서 실행하는 경우 각 파일의 코드를 셀 단위로 실행해도 됩니다.

2. requirements.txt

```
# PyTorch 딥러닝 프레임워크입니다.  
torch  
  
# Hugging Face 사전 학습 모델과 Tokenizer를 사용합니다.  
transformers  
  
# JSON, CSV, Hugging Face 데이터셋을 처리합니다.  
datasets  
  
# Supervised Fine-Tuning을 위한 SFTTrainer를 제공합니다.
```

```
trl

# LoRA, QLoRA 등 PEFT 학습 기능을 제공합니다.
peft

# 다중 GPU 및 혼합 정밀도 학습을 지원합니다.
accelerate

# 모델을 8bit 또는 4bit로 양자화하여 GPU 메모리를 절약합니다.
bitsandbytes

# 환경변수 파일인 .env를 읽을 때 사용합니다.
python-dotenv

# 학습 진행 상황과 손실값을 시각화할 때 사용할 수 있습니다.
tensorboard

# 모델 및 데이터셋 다운로드 기능을 제공합니다.
huggingface-hub

# SentencePiece 기반 Tokenizer가 필요한 모델을 지원합니다.
sentencepiece
```

설치 명령은 다음과 같습니다.

```
pip install -U pip setuptools wheel
```

```
pip install -r requirements.txt
```

Notebook 환경에서는 다음과 같이 실행합니다.

```
# Notebook에서 필요한 라이브러리를 한 번에 설치합니다.
```

```
!pip install -U torch transformers datasets trl peft accelerate bitsandbytes \
python-dotenv tensorboard huggingface-hub sentencepiece
```

3. .env.example

```
# Hugging Face 로그인에 필요한 모델을 사용할 때 토큰을 입력합니다.  
# 공개 모델만 사용한다면 입력하지 않아도 됩니다.  
HF_TOKEN=  
  
# 학습할 Hugging Face 모델의 이름입니다.  
MODEL_NAME=Qwen/Qwen2.5-0.5B-Instruct  
  
# 학습된 LoRA Adapter가 저장될 경로입니다.  
OUTPUT_DIR=outputs/qwen2.5-korean-sft-lora
```

실제 사용 시 파일 이름을 .env로 변경합니다.

4. 학습 데이터 생성 코드

01_create_dataset.py

```
"""  
한국어 상담용 Supervised Fine-Tuning 데이터셋을 생성하는 파일입니다.  
  
생성되는 데이터 형식은 다음과 같습니다.  
  
{  
    "messages": [  
        {"role": "system", "content": "시스템 지시사항"},  
        {"role": "user", "content": "사용자 질문"},  
        {"role": "assistant", "content": "학습 대상 답변"}  
    ]  
}  
"""  
  
# json 모듈은 파이썬 객체를 JSON 문자열로 변환하거나  
# JSON 문자열을 파이썬 객체로 변환할 때 사용합니다.  
import json
```

```

# Path 클래스는 운영체제에 독립적인 방식으로
# 파일과 디렉터리 경로를 처리할 수 있게 합니다.
from pathlib import Path

# random 모듈은 데이터의 순서를 무작위로 섞을 때 사용합니다.
import random

# 현재 파일이 있는 디렉터리를 프로젝트의 기준 디렉터리로 지정합니다.
BASE_DIR = Path(__file__).resolve().parent

# 학습 데이터가 저장될 data 디렉터리 경로를 설정합니다.
DATA_DIR = BASE_DIR / "data"

# 학습 데이터 파일 경로를 지정합니다.
TRAIN_FILE = DATA_DIR / "train.jsonl"

# 검증 데이터 파일 경로를 지정합니다.
VALID_FILE = DATA_DIR / "valid.jsonl"

# 동일한 결과를 재현할 수 있도록 난수 시드를 고정합니다.
RANDOM_SEED = 42

# 모델에게 공통으로 적용할 시스템 프롬프트를 정의합니다.
SYSTEM_PROMPT = """
당신은 온라인 쇼핑몰의 고객 상담 AI입니다.
고객의 질문을 정확하게 이해하고 친절하고 간결하게 답변합니다.
확인되지 않은 정책이나 정보를 임의로 만들지 않습니다.
추가 정보가 필요한 경우 고객에게 필요한 정보를 요청합니다.
답변은 원칙적으로 한국어로 작성합니다.
""".strip()

# SFT 학습에 사용할 상담 데이터를 리스트로 정의합니다.
# 각 항목에서 user는 모델 입력이고 assistant는 학습할 정답입니다.
RAW_SAMPLES = [

```

```

{
  "user": "주문한 상품을 취소하고 싶습니다.",
  "assistant": (
    "상품이 아직 배송 준비 전이라면 주문 내역에서 직접 취소할 수 있습니다. "
    "이미 배송이 시작된 경우에는 상품 수령 후 반품을 신청해야 합니다. "
    "정확한 상태를 확인하려면 주문번호를 알려주세요."
  ),
},
{
  "user": "배송은 보통 얼마나 걸리나요?",
  "assistant": (
    "일반 배송은 결제 완료 후 평균 2~3영업일 정도 소요됩니다. "
    "도서산간 지역이나 주문량이 많은 기간에는 배송이 지연될 수 있습니다."
  ),
},
{
  "user": "상품을 받았는데 파손되어 있습니다.",
  "assistant": (
    "불편을 드려 죄송합니다. 파손된 상품과 포장 상태를 확인할 수 있는 사진을 준비한 뒤 "
    "주문번호와 함께 교환 또는 반품을 신청해 주세요. "
    "상품 확인 후 교환이나 환불 절차를 안내해 드리겠습니다."
  ),
},
{
  "user": "반품 배송비는 누가 부담하나요?",
  "assistant": (
    "단순 변심에 의한 반품은 일반적으로 고객이 반품 배송비를 부담합니다. "
    "상품 불량, 오배송 또는 파손과 같이 판매자 책임이 있는 경우에는 "
    "판매자가 반품 배송비를 부담합니다."
  ),
},
{
  "user": "환불은 언제 처리되나요?",
  "assistant": (
    "반품 상품이 판매자에게 도착하고 검수가 완료된 후 환불이 진행됩니다. "
    "카드 결제는 카드사에 따라 승인 취소 반영까지 추가로 3~7영업일이 걸릴 수 있습니다."
  ),
},

```



```

    },
    {
      "user": "주문번호를 모르는데 주문 조회가 가능한가요?",
      "assistant": (
        "회원 주문이라면 로그인 후 주문 내역에서 확인할 수 있습니다. "
        "비회원 주문이라면 주문자 이름, 연락처 또는 결제 정보를 이용해 조회할 수 있습니다."
      ),
    },
    {
      "user": "결제는 완료됐는데 주문 상태가 결제 대기로 나옵니다.",
      "assistant": (
        "결제 정보가 주문 시스템에 반영되는 데 일시적으로 시간이 걸릴 수 있습니다. "
        "잠시 후 주문 내역을 다시 확인해 주세요. "
        "계속 결제 대기로 표시된다면 주문번호와 결제 시간을 알려주세요."
      ),
    },
    {
      "user": "배송지를 잘못 입력했습니다.",
      "assistant": (
        "상품이 배송 준비 전이라면 주문 내역 또는 고객센터를 통해 배송지 변경을 요청할 수 있습니다. "
        "이미 출고된 경우에는 배송지 변경이 제한될 수 있으므로 빠르게 문의해 주세요."
      ),
    },
    {
      "user": "교환하고 싶은데 어떻게 해야 하나요?",
      "assistant": (
        "주문 내역에서 교환할 상품을 선택한 후 교환 사유와 원하는 옵션을 입력해 신청해 주세요. "
        "상품 상태 확인 후 회수 및 재배송 절차가 진행됩니다."
      ),
    },
    {
      "user": "회원 탈퇴 후 주문 내역도 삭제되나요?",
      "assistant": (
        "회원 탈퇴 후에도 전자상거래 관련 법령에 따라 거래 기록 일부는 일정 기간 보관될 수 있습니다. "

```

```

        "보관 기간이 종료되면 관련 정책에 따라 안전하게 삭제됩니다."
    ),
},
{
    "user": "쿠폰을 사용하지 못하고 결제했습니다.",
    "assistant": (
        "결제가 완료된 주문에는 일반적으로 쿠폰을 추가 적용할 수 없습니다. "
        "상품이 아직 배송 준비 전이라면 기존 주문을 취소한 뒤 쿠폰을 적용하여 다시 주문할 수 있습니다."
    ),
},
{
    "user": "품절된 상품은 언제 다시 입고되나요?",
    "assistant": (
        "재입고 일정은 상품과 공급 상황에 따라 달라집니다. "
        "상품 페이지에 재입고 알림 기능이 있다면 신청해 주세요. "
        "정확한 확인을 위해 상품명을 알려주시면 확인 방법을 안내해 드리겠습니다."
    ),
},
{
    "user": "다른 상품이 배송되었습니다.",
    "assistant": (
        "오배송으로 불편을 드려 죄송합니다. "
        "받으신 상품의 사진과 주문번호를 준비하여 교환 또는 반품을 신청해 주세요. "
        "확인 후 올바른 상품을 받을 수 있도록 안내해 드리겠습니다."
    ),
},
{
    "user": "현금영수증을 발급받고 싶습니다.",
    "assistant": (
        "현금 결제 과정에서 현금영수증 발급 정보를 입력할 수 있습니다. "
        "결제 후 발급이 필요한 경우 주문번호와 결제 정보를 확인한 뒤 고객센터에 요청해 주세요."
    ),
},
{
    "user": "고객센터 운영 시간이 궁금합니다.",

```

```

"assistant": (
    "고객센터 운영 시간은 쇼핑몰 정책에 따라 다를 수 있습니다. "
    "사이트의 고객센터 또는 이용 안내 페이지에서 정확한 운영 시간을 확인해 주세요."
),
},
{
"user": "상품 사용 후에도 반품할 수 있나요?",
"assistant": (
    "상품을 사용하여 가치가 감소했거나 재판매가 어려운 상태라면 반품이 제한될 수 있습니다. "
    "다만 상품 자체의 하자가 확인되는 경우에는 별도 절차로 처리될 수 있습니다."
),
},
{
"user": "배송 완료라고 나오는데 상품을 받지 못했습니다.",
"assistant": (
    "먼저 가족, 경비실, 무인 택배함 또는 문 앞에 상품이 보관되었는지 확인해 주세요. "
    "상품이 확인되지 않으면 배송 조회에 표시된 택배사와 기사에게 문의한 뒤 "
    "주문번호를 고객센터에 알려주세요."
),
},
{
"user": "결제 수단을 변경하고 싶습니다.",
"assistant": (
    "결제가 완료된 주문의 결제 수단은 직접 변경하기 어렵습니다. "
    "배송 준비 전이라면 주문을 취소한 후 원하는 결제 수단으로 다시 주문해 주세요."
),
},
{
"user": "부분 환불도 가능한가요?",
"assistant": (
    "여러 상품을 함께 주문했다면 일부 상품만 선택하여 반품 및 환불을 신청할 수 있습니다. "
    "다만 할인이나 쿠폰 조건이 변경되면 최종 환불 금액이 달라질 수 있습니다."
),
},
{

```

```

        "user": "문의 글을 남겼는데 답변이 없습니다.",
        "assistant": (
            "문의량이 많으면 답변이 지연될 수 있습니다. "
            "문의 내역에서 접수 상태를 확인해 주세요. "
            "오랫동안 답변이 없다면 문의 날짜와 제목을 알려주시면 확인 방법을 안내해 드리겠습니다."
        ),
    },
]

```

```
def convert_to_messages(sample: dict) -> dict:
```

```
    """
```

```

    user와 assistant로 구성된 원본 샘플을
    Hugging Face 대화형 messages 형식으로 변환합니다.

```

```
    Args:
```

```
        sample: user와 assistant 키를 가진 원본 데이터입니다.
```

```
    Returns:
```

```
        system, user, assistant 메시지를 포함한 딕셔너리입니다.
```

```
    """
```

```
    # SFTTrainer가 인식할 수 있도록 messages 키를 사용합니다.
```

```
    return {
```

```
        "messages": [
```

```
            # system 메시지는 모델의 역할과 답변 원칙을 지정합니다.
```

```
            {
```

```
                "role": "system",
```

```
                "content": SYSTEM_PROMPT,
```

```
            },
```

```
            # user 메시지는 모델이 입력으로 받는 고객 질문입니다.
```

```
            {
```

```
                "role": "user",
```

```
                "content": sample["user"],
```

```
            },
```

```

        # assistant 메시지는 모델이 학습해야 하는 정답입니다.
        {
            "role": "assistant",
            "content": sample["assistant"],
        },
    ]
}

```

def validate_sample(sample: dict) -> None:

"""

한 개의 학습 데이터가 올바른 messages 구조인지 검사합니다.

구조에 문제가 있으면 ValueError를 발생시켜
잘못된 데이터가 학습에 사용되는 것을 방지합니다.

"""

최상위 객체에 messages 키가 존재하는지 검사합니다.

if "messages" not in sample:

raise ValueError("데이터에 messages 키가 없습니다.")

messages 값이 리스트인지 검사합니다.

if not isinstance(sample["messages"], list):

raise ValueError("messages 값은 리스트여야 합니다.")

system, user, assistant 메시지 3개가 있는지 검사합니다.

if len(sample["messages"]) != 3:

raise ValueError("각 데이터는 system, user, assistant 메시지를 포함해야 합니다.")

예상하는 역할 순서를 리스트로 정의합니다.

expected_roles = ["system", "user", "assistant"]

실제 메시지와 예상 역할을 한 개씩 비교합니다.

for message, expected_role in zip(sample["messages"], expected_roles):

각 메시지가 딕셔너리인지 검사합니다.

```

if not isinstance(message, dict):
    raise ValueError("각 메시지는 딕셔너리여야 합니다.")

# role 값이 예상 역할과 일치하는지 검사합니다.
if message.get("role") != expected_role:
    raise ValueError(
        f"잘못된 역할 순서입니다. 예상={expected_role}, 실제={message.get('role')}"
    )

# content 값이 비어 있지 않은 문자열인지 검사합니다.
content = message.get("content")

if not isinstance(content, str) or not content.strip():
    raise ValueError(f"{expected_role} 메시지의 content가 비어 있습니다.")

def save_jsonl(samples: list[dict], file_path: Path) -> None:
    """
    데이터 리스트를 JSON Lines 형식으로 저장합니다.

    JSON Lines는 한 줄에 JSON 객체 하나를 저장하는 형식으로,
    대규모 학습 데이터를 순차적으로 처리하기 편리합니다.
    """

    # UTF-8 인코딩으로 출력 파일을 엽니다.
    # ensure_ascii=False와 함께 사용하면 한글이 그대로 저장됩니다.
    with file_path.open("w", encoding="utf-8") as file:

        # 데이터 한 개씩 반복합니다.
        for sample in samples:

            # 파이썬 딕셔너리를 JSON 문자열로 변환합니다.
            json_line = json.dumps(
                sample,
                ensure_ascii=False,
            )

```

```

        # JSON 객체 하나를 한 줄에 기록합니다.
        file.write(json_line + "\n")

def main() -> None:
    """
    데이터 변환, 검증, 분할 및 저장을 수행하는 메인 함수입니다.
    """

    # data 디렉터리가 없으면 새로 생성합니다.
    # parents=True는 상위 디렉터리까지 생성합니다.
    # exist_ok=True는 이미 존재해도 오류를 발생시키지 않습니다.
    DATA_DIR.mkdir(
        parents=True,
        exist_ok=True,
    )

    # 원본 데이터를 messages 형식으로 변환합니다.
    converted_samples = [
        convert_to_messages(sample)
        for sample in RAW_SAMPLES
    ]

    # 변환된 모든 데이터를 검사합니다.
    for sample in converted_samples:
        validate_sample(sample)

    # 동일한 데이터 분할 결과가 나오도록 Random 객체를 생성합니다.
    random_generator = random.Random(RANDOM_SEED)

    # 원본 목록을 변경하지 않도록 복사본을 만듭니다.
    shuffled_samples = converted_samples.copy()

    # 데이터 순서를 무작위로 섞습니다.
    random_generator.shuffle(shuffled_samples)

    # 전체 데이터의 80%를 학습 데이터로 사용합니다.

```

```

split_index = int(len(shuffled_samples) * 0.8)

# 앞부분을 학습 데이터로 분리합니다.
train_samples = shuffled_samples[:split_index]

# 뒷부분을 검증 데이터로 분리합니다.
valid_samples = shuffled_samples[split_index:]

# 학습 데이터를 train.jsonl로 저장합니다.
save_jsonl(
    train_samples,
    TRAIN_FILE,
)

# 검증 데이터를 valid.jsonl로 저장합니다.
save_jsonl(
    valid_samples,
    VALID_FILE,
)

# 생성 결과를 화면에 출력합니다.
print("=" * 70)
print("SFT 데이터셋 생성 완료")
print("=" * 70)
print(f"전체 데이터 수 : {len(shuffled_samples)}")
print(f"학습 데이터 수 : {len(train_samples)}")
print(f"검증 데이터 수 : {len(valid_samples)}")
print(f"학습 파일      : {TRAIN_FILE}")
print(f"검증 파일      : {VALID_FILE}")

# 이 파일을 직접 실행한 경우에만 main 함수를 호출합니다.
if __name__ == "__main__":
    main()

```


실행합니다.

python 01_create_dataset.py

5. SFT 학습 전체 코드

02_train_sft.py

이 코드는 다음 기능을 모두 포함합니다.

CUDA 확인

→ 데이터셋 로드

→ Tokenizer 로드

→ 4bit 양자화

→ QLoRA 준비

→ LoRA 설정

→ SFTTrainer 생성

→ 학습 실행

→ 평가

→ Adapter 저장

→ 학습 이력 저장

→ 간단한 추론 테스트

4비트 양자화 모델 위에 LoRA Adapter를 추가하는 방식은 일반적으로 QLoRA라고 합니다. PEFT 공식 문서에서는 양자화 모델을 학습하기 전에 `prepare_model_for_kbit_training()`을 호출하는 구성을 안내합니다. ([Hugging Face](#))

```
"""
Qwen2.5-0.5B-Instruct 모델을 대상으로
한국어 고객 상담 데이터 SFT를 수행하는 전체 학습 코드입니다.

학습 방식:
    Supervised Fine-Tuning
    + 4bit Quantization
    + LoRA
    = QLoRA 기반 SFT
"""
```

```
# os 모듈은 환경변수 설정과 운영체제 기능을 사용할 때 필요합니다.
import os

# json 모듈은 학습 결과와 설정 정보를 JSON 파일로 저장할 때 사용합니다.
import json

# gc 모듈은 사용하지 않는 파이썬 객체를 정리하여
# 메모리를 확보하는 데 사용합니다.
import gc

# Path는 파일과 디렉터리 경로를 안전하게 처리합니다.
from pathlib import Path

# Any는 다양한 자료형을 허용하는 타입 힌트에 사용합니다.
from typing import Any

# PyTorch는 모델 학습과 GPU 연산을 담당합니다.
import torch

# .env 파일의 환경변수를 불러오기 위해 사용합니다.
from dotenv import load_dotenv

# Hugging Face datasets의 JSON 데이터 로더입니다.
from datasets import load_dataset

# Hugging Face 모델, Tokenizer 및 양자화 설정 클래스를 불러옵니다.
from transformers import (
    AutoModelForCausalLM,
    AutoTokenizer,
    BitsAndBytesConfig,
    set_seed,
)

# LoRA 설정과 k-bit 학습 준비 함수를 불러옵니다.
from peft import (
    LoraConfig,
    prepare_model_for_kbit_training,
```

```

)

# TRL에서 SFT 전용 설정과 Trainer를 불러옵니다.
from trl import (
    SFTConfig,
    SFTTrainer,
)

# -----
# 1. 기본 경로 설정
# -----

# 현재 파이썬 파일이 위치한 디렉터리를 프로젝트 기준 경로로 지정합니다.
BASE_DIR = Path(__file__).resolve().parent

# 프로젝트 루트의 .env 파일을 읽습니다.
load_dotenv(BASE_DIR / ".env")

# 학습 데이터 파일 경로를 설정합니다.
TRAIN_FILE = BASE_DIR / "data" / "train.jsonl"

# 검증 데이터 파일 경로를 설정합니다.
VALID_FILE = BASE_DIR / "data" / "valid.jsonl"

# 환경변수에 MODEL_NAME이 있으면 해당 값을 사용하고,
# 없으면 Qwen2.5-0.5B-Instruct를 기본 모델로 사용합니다.
MODEL_NAME = os.getenv(
    "MODEL_NAME",
    "Qwen/Qwen2.5-0.5B-Instruct",
)

# 환경변수에 OUTPUT_DIR이 있으면 해당 경로를 사용합니다.
OUTPUT_DIR = BASE_DIR / os.getenv(
    "OUTPUT_DIR",
    "outputs/qwen2.5-korean-sft-lora",
)

```

```

# Trainer가 중간 체크포인트를 저장할 경로입니다.
CHECKPOINT_DIR = BASE_DIR / "outputs" / "checkpoints"

# Hugging Face 접근 토큰을 환경변수에서 읽습니다.
HF_TOKEN = os.getenv("HF_TOKEN") or None

# 난수 시드를 고정하여 가능한 범위에서 학습 결과를 재현합니다.
SEED = 42

# -----
# 2. 학습 하이퍼파라미터
# -----

# 한 번에 모델에 입력할 최대 토큰 수입니다.
MAX_LENGTH = 512

# GPU에 한 번에 올릴 학습 데이터 개수입니다.
# GPU 메모리가 부족하면 1로 유지합니다.
TRAIN_BATCH_SIZE = 1

# 평가 과정에서 GPU에 한 번에 올릴 데이터 개수입니다.
EVAL_BATCH_SIZE = 1

# 여러 미니 배치의 Gradient를 누적할 횟수입니다.
# 실제 배치 크기는 TRAIN_BATCH_SIZE × GRADIENT_ACCUMULATION_STEPS입니다.
GRADIENT_ACCUMULATION_STEPS = 8

# 전체 학습 데이터를 반복할 횟수입니다.
NUM_TRAIN_EPOCHS = 3

# LoRA 학습에서 사용할 학습률입니다.
LEARNING_RATE = 2e-4

# 가중치가 과도하게 커지는 것을 억제하기 위한 Weight Decay입니다.
WEIGHT_DECAY = 0.01

```

```

# 학습률을 처음부터 바로 크게 적용하지 않고
# 점진적으로 증가시키는 Warm-up 비율입니다.
WARMUP_RATIO = 0.05

# 몇 Step마다 학습 상태와 Loss를 출력할지 지정합니다.
LOGGING_STEPS = 1

# 최대 몇 개의 체크포인트를 유지할지 지정합니다.
SAVE_TOTAL_LIMIT = 2

def print_environment() -> None:
    """
    현재 실행 환경의 Python, PyTorch, CUDA 및 GPU 정보를 출력합니다.
    """

    # 실행 환경 구분선을 출력합니다.
    print("=" * 80)
    print("1. 실행 환경 확인")
    print("=" * 80)

    # 현재 설치된 PyTorch 버전을 출력합니다.
    print(f"PyTorch 버전      : {torch.__version__}")

    # CUDA 사용 가능 여부를 출력합니다.
    print(f"CUDA 사용 가능      : {torch.cuda.is_available()}")

    # CUDA를 사용할 수 있는 경우 GPU 정보를 추가로 출력합니다.
    if torch.cuda.is_available():

        # 사용 가능한 GPU 개수를 출력합니다.
        print(f"GPU 개수              : {torch.cuda.device_count()}")

        # 첫 번째 GPU의 이름을 출력합니다.
        print(f"GPU 이름              : {torch.cuda.get_device_name(0)}")

```

```

# 현재 PyTorch가 사용하는 CUDA 버전을 출력합니다.
print(f"PyTorch CUDA 버전 : {torch.version.cuda}")

# 전체 GPU 메모리를 Byte 단위에서 GB 단위로 변환합니다.
total_memory_gb = (
    torch.cuda.get_device_properties(0).total_memory
    / 1024**3
)

# 전체 GPU 메모리를 출력합니다.
print(f"GPU 전체 메모리 : {total_memory_gb:.2f} GB")

else:
    # QLoRA 학습은 NVIDIA GPU 환경에서 실행할 것을 안내합니다.
    print(
        "경고: CUDA GPU가 감지되지 않았습니다. "
        "4bit QLoRA 학습은 NVIDIA GPU 환경에서 실행해야 합니다."
    )

def validate_files() -> None:
    """
    학습 전에 필요한 데이터 파일이 존재하는지 확인합니다.
    """

    # 학습 데이터가 존재하지 않으면 FileNotFoundError를 발생시킵니다.
    if not TRAIN_FILE.exists():
        raise FileNotFoundError(
            f"학습 데이터가 없습니다: {TRAIN_FILE}\n"
            "먼저 python 01_create_dataset.py를 실행하세요."
        )

    # 검증 데이터가 존재하지 않으면 FileNotFoundError를 발생시킵니다.
    if not VALID_FILE.exists():
        raise FileNotFoundError(
            f"검증 데이터가 없습니다: {VALID_FILE}\n"
            "먼저 python 01_create_dataset.py를 실행하세요."
        )

```

```

    )

def load_sft_datasets():
    """
    train.jsonl과 valid.jsonl 파일을
    Hugging Face Dataset 객체로 읽어옵니다.

    Returns:
        train_dataset: 학습 데이터셋
        valid_dataset: 검증 데이터셋
    """

    # JSONL 파일 경로를 train과 validation 이름으로 연결합니다.
    data_files = {
        "train": str(TRAIN_FILE),
        "validation": str(VALID_FILE),
    }

    # JSON 데이터셋을 Hugging Face DatasetDict 형식으로 로드합니다.
    dataset_dict = load_dataset(
        "json",
        data_files=data_files,
    )

    # train 분할을 학습 데이터셋으로 가져옵니다.
    train_dataset = dataset_dict["train"]

    # validation 분할을 검증 데이터셋으로 가져옵니다.
    valid_dataset = dataset_dict["validation"]

    # 데이터셋 정보를 출력합니다.
    print("\n" + "=" * 80)
    print("2. 데이터셋 로드")
    print("=" * 80)
    print(f"학습 데이터 수 : {len(train_dataset)}")
    print(f"검증 데이터 수 : {len(valid_dataset)}")

```

```

print(f"데이터 컬럼      : {train_dataset.column_names}")

# 첫 번째 학습 샘플을 확인합니다.
print("\n첫 번째 학습 데이터:")
print(
    json.dumps(
        train_dataset[0],
        ensure_ascii=False,
        indent=2,
    )
)

# 학습 및 검증 데이터셋을 반환합니다.
return train_dataset, valid_dataset

def select_compute_dtype() -> torch.dtype:
    """
    GPU가 bfloat16 연산을 지원하면 bfloat16을 사용하고,
    지원하지 않으면 float16을 사용합니다.

    Returns:
        선택된 PyTorch 데이터 타입
    """

    # CUDA가 있고 GPU가 BF16을 지원하는지 검사합니다.
    if (
        torch.cuda.is_available()
        and torch.cuda.is_bf16_supported()
    ):
        # Ampere 계열 이상 GPU에서는 일반적으로 BF16을 사용할 수 있습니다.
        return torch.bfloat16

    # BF16을 사용할 수 없으면 FP16을 사용합니다.
    return torch.float16

```



```

def load_tokenizer():
    """
    사전 학습 모델의 Tokenizer를 불러오고
    Padding 관련 설정을 적용합니다.

    Returns:
        설정이 완료된 Tokenizer
    """

    # 모델 이름과 Tokenizer 로드 시작 정보를 출력합니다.
    print("\n" + "=" * 80)
    print("3. Tokenizer 로드")
    print("=" * 80)
    print(f"Tokenizer 모델 : {MODEL_NAME}")

    # Hugging Face Hub에서 모델에 맞는 Tokenizer를 불러옵니다.
    tokenizer = AutoTokenizer.from_pretrained(
        MODEL_NAME,

        # 원격 저장소에 사용자 정의 코드가 있을 경우 사용할 수 있게 합니다.
        trust_remote_code=True,

        # 비공개 또는 사용 승인이 필요한 모델에 접근할 때 토큰을 사용합니다.
        token=HF_TOKEN,
    )

    # 일부 모델은 pad_token이 지정되지 않을 수 있습니다.
    if tokenizer.pad_token is None:

        # 문장 종료 토큰을 Padding 토큰으로 사용합니다.
        tokenizer.pad_token = tokenizer.eos_token

    # Causal Language Model 학습에서는 오른쪽 Padding을 사용합니다.
    tokenizer.padding_side = "right"

    # 모델 설정에 사용할 pad_token_id를 확인합니다.
    print(f"PAD Token          : {tokenizer.pad_token}")

```

```

print(f"PAD Token ID      : {tokenizer.pad_token_id}")
print(f"EOS Token        : {tokenizer.eos_token}")
print(f"EOS Token ID     : {tokenizer.eos_token_id}")

# 설정된 Tokenizer를 반환합니다.
return tokenizer

def create_quantization_config(
    compute_dtype: torch.dtype,
) -> BitsAndBytesConfig:
    """
    4bit QLoRA 학습을 위한 BitsAndBytesConfig를 생성합니다.

    Args:
        compute_dtype: 실제 계산에 사용할 FP16 또는 BF16 타입입니다.

    Returns:
        4bit 양자화 설정 객체
    """

    # BitsAndBytes의 4bit 양자화 설정을 생성합니다.
    quantization_config = BitsAndBytesConfig(

        # 모델 가중치를 4bit로 로드합니다.
        load_in_4bit=True,

        # QLoRA에서 일반적으로 사용하는 NF4 양자화 방식을 적용합니다.
        bnb_4bit_quant_type="nf4",

        # 양자화 상수를 다시 양자화하여 메모리 사용량을 추가로 줄입니다.
        bnb_4bit_use_double_quant=True,

        # 행렬 연산은 FP16 또는 BF16으로 수행합니다.
        bnb_4bit_compute_dtype=compute_dtype,
    )

```

```

# 생성한 양자화 설정을 반환합니다.
return quantization_config

def load_quantized_model(
    tokenizer,
    compute_dtype: torch.dtype,
):
    """
    사전 학습 모델을 4bit로 불러오고
    k-bit 학습이 가능하도록 전처리합니다.

    Args:
        tokenizer: 모델 Tokenizer
        compute_dtype: 연산에 사용할 데이터 타입

    Returns:
        QLoRA 학습 준비가 완료된 모델
    """

    # 모델 로드 정보를 출력합니다.
    print("\n" + "=" * 80)
    print("4. 4bit 사전 학습 모델 로드")
    print("=" * 80)
    print(f"모델 이름           : {MODEL_NAME}")
    print(f"계산 데이터 타입      : {compute_dtype}")

    # 앞에서 정의한 함수로 4bit 양자화 설정을 생성합니다.
    quantization_config = create_quantization_config(
        compute_dtype=compute_dtype,
    )

    # Hugging Face Hub에서 Causal Language Model을 불러옵니다.
    model = AutoModelForCausalLM.from_pretrained(
        MODEL_NAME,

        # 4bit 양자화 설정을 전달합니다.

```

```

quantization_config=quantization_config,

# 단일 GPU 환경에서 모델을 첫 번째 GPU에 배치합니다.
device_map={"": 0},

# 원격 저장소의 사용자 정의 모델 코드를 허용합니다.
trust_remote_code=True,

# 비공개 모델 접근용 Hugging Face 토큰입니다.
token=HF_TOKEN,
)

# Tokenizer와 동일한 Padding Token ID를 모델 설정에도 지정합니다.
model.config.pad_token_id = tokenizer.pad_token_id

# 학습 과정에서 캐시를 사용하면 Gradient Checkpointing과
# 충돌할 수 있으므로 캐시 사용을 비활성화합니다.
model.config.use_cache = False

# 양자화된 모델을 LoRA 학습에 적합한 형태로 준비합니다.
model = prepare_model_for_kbit_training(
    model,

    # Gradient Checkpointing을 사용할 수 있도록 입력 Gradient를 준비합니다.
    use_gradient_checkpointing=True,
)

# 모델의 Gradient Checkpointing 기능을 활성화합니다.
model.gradient_checkpointing_enable()

# 준비된 모델을 반환합니다.
return model

def create_lora_config() -> LoraConfig:
    """
    Qwen 계열 Causal Language Model에 적용할

```

LoRA Adapter 설정을 생성합니다.

Returns:

LoraConfig 객체

"""

LoRA 설정 생성 시작을 출력합니다.

```
print("\n" + "=" * 80)
```

```
print("5. LoRA 설정")
```

```
print("=" * 80)
```

LoRA Adapter의 세부 설정을 정의합니다.

```
lora_config = LoraConfig(
```

```
    # 저차원 행렬의 Rank입니다.
```

```
    # 값이 크면 표현력은 증가하지만 학습 파라미터와 메모리도 증가합니다.
```

```
    r=16,
```

```
    # LoRA 출력에 적용되는 Scaling 계수입니다.
```

```
    lora_alpha=32,
```

```
    # 과적합을 줄이기 위해 LoRA 경로에 적용할 Dropout 비율입니다.
```

```
    lora_dropout=0.05,
```

```
    # 기존 Linear Layer의 Bias는 학습하지 않습니다.
```

```
    bias="none",
```

```
    # 현재 모델이 다음 토큰을 생성하는 Causal LM임을 지정합니다.
```

```
    task_type="CAUSAL_LM",
```

```
    # Qwen Transformer 내부에서 LoRA를 적용할 Linear Layer입니다.
```

```
    target_modules=[
```

```
        # Query Projection Layer입니다.
```

```
        "q_proj",
```

```
        # Key Projection Layer입니다.
```

```
        "k_proj",
```

```

        # Value Projection Layer입니다.
        "v_proj",

        # Attention Output Projection Layer입니다.
        "o_proj",

        # Feed Forward Network의 Gate Layer입니다.
        "gate_proj",

        # Feed Forward Network의 확장 Projection Layer입니다.
        "up_proj",

        # Feed Forward Network의 축소 Projection Layer입니다.
        "down_proj",
    ],
)

# 주요 LoRA 설정값을 출력합니다.
print(f"LoRA Rank          : {lora_config.r}")
print(f"LoRA Alpha          : {lora_config.lora_alpha}")
print(f"LoRA Dropout          : {lora_config.lora_dropout}")
print(f"Target Modules        : {lora_config.target_modules}")

# 생성된 LoRA 설정을 반환합니다.
return lora_config

```

```

def create_training_config(
    compute_dtype: torch.dtype,
) -> SFTConfig:
    """
    SFTTrainer가 사용할 학습 하이퍼파라미터를 생성합니다.

    Args:
        compute_dtype: BF16 또는 FP16 중 사용할 연산 타입
    """

```

Returns:

SFTConfig 객체

"""

BF16 사용 여부를 판단합니다.

use_bf16 = compute_dtype == torch.bfloat16

FP16 사용 여부를 판단합니다.

use_fp16 = compute_dtype == torch.float16

출력 디렉터리가 없으면 생성합니다.

OUTPUT_DIR.mkdir(

parents=True,

exist_ok=True,

)

체크포인트 디렉터리가 없으면 생성합니다.

CHECKPOINT_DIR.mkdir(

parents=True,

exist_ok=True,

)

SFT 학습 설정을 생성합니다.

training_config = SFTConfig(

체크포인트와 Trainer 결과를 저장할 디렉터리입니다.

output_dir=str(CHECKPOINT_DIR),

전체 학습 Epoch 수입니다.

num_train_epochs=NUM_TRAIN_EPOCHS,

GPU 한 개당 학습 배치 크기입니다.

per_device_train_batch_size=TRAIN_BATCH_SIZE,

GPU 한 개당 평가 배치 크기입니다.

per_device_eval_batch_size=EVAL_BATCH_SIZE,

```
# Gradient를 누적할 Step 수입니다.
gradient_accumulation_steps=GRADIENT_ACCUMULATION_STEPS,

# LoRA 파라미터를 갱신할 학습률입니다.
learning_rate=LEARNING_RATE,

# 가중치 감쇠 계수입니다.
weight_decay=WEIGHT_DECAY,

# 학습률 Warm-up 비율입니다.
warmup_ratio=WARMUP_RATIO,

# Cosine 방식으로 학습률을 감소시킵니다.
lr_scheduler_type="cosine",

# AdamW 계열 Optimizer를 사용합니다.
optim="paged_adamw_8bit",

# 몇 Step마다 로그를 출력할지 지정합니다.
logging_steps=LOGGING_STEPS,

# 학습 로그를 첫 Step부터 기록합니다.
logging_first_step=True,

# 매 Epoch가 끝날 때 검증 데이터로 평가합니다.
eval_strategy="epoch",

# 매 Epoch가 끝날 때 체크포인트를 저장합니다.
save_strategy="epoch",

# 최대 체크포인트 보관 개수입니다.
save_total_limit=SAVE_TOTAL_LIMIT,

# 학습이 끝난 후 가장 좋은 모델을 자동으로 불러옵니다.
load_best_model_at_end=True,

# 최적 모델 선택 기준으로 validation loss를 사용합니다.
```



```
metric_for_best_model="eval_loss",

# Loss는 낮을수록 좋은 값이므로 greater_is_better를 False로 지정합니다.
greater_is_better=False,

# BF16 사용 여부를 설정합니다.
bf16=use_bf16,

# FP16 사용 여부를 설정합니다.
fp16=use_fp16,

# Gradient Checkpointing으로 GPU 메모리 사용량을 줄입니다.
gradient_checkpointing=True,

# Gradient Checkpointing에서 비재진입 방식을 사용합니다.
gradient_checkpointing_kwargs={
    "use_reentrant": False,
},

# 모델에 입력할 최대 토큰 길이입니다.
max_length=MAX_LENGTH,

# 길이가 다른 문장을 하나로 묶어 Padding을 줄이는 옵션입니다.
# 작은 실습 데이터에서는 False로 두는 것이 이해하기 쉽습니다.
packing=False,

# TensorBoard 형식으로 학습 로그를 저장합니다.
report_to=["tensorboard"],

# 데이터셋 처리 시 사용할 프로세스 수입니다.
dataset_num_proc=1,

# Trainer가 데이터 컬럼을 임의로 제거하지 않도록 합니다.
remove_unused_columns=False,

# 난수 시드를 지정합니다.
seed=SEED,
```

```

        # 데이터 샘플링 관련 난수 시드입니다.
        data_seed=SEED,
    )

    # 학습 설정을 반환합니다.
    return training_config

def count_parameters(model) -> dict[str, Any]:
    """
    전체 파라미터와 실제 학습 가능한 파라미터 개수를 계산합니다.

    Args:
        model: 파라미터를 확인할 모델

    Returns:
        전체, 학습 가능, 학습 비율을 포함한 딕셔너리
    """

    # 전체 파라미터 개수의 누적값입니다.
    total_parameters = 0

    # requires_grad=True인 학습 가능 파라미터의 누적값입니다.
    trainable_parameters = 0

    # 모델의 모든 파라미터를 반복합니다.
    for parameter in model.parameters():

        # 현재 Tensor가 가진 전체 원소 수를 전체 파라미터에 더합니다.
        total_parameters += parameter.numel()

        # Gradient 계산 대상이면 학습 가능 파라미터에 더합니다.
        if parameter.requires_grad:
            trainable_parameters += parameter.numel()

    # 전체 대비 학습 가능 파라미터 비율을 계산합니다.

```

```

trainable_ratio = (
    100 * trainable_parameters / total_parameters
    if total_parameters > 0
    else 0.0
)

# 계산 결과를 딕셔너리로 반환합니다.
return {
    "total_parameters": total_parameters,
    "trainable_parameters": trainable_parameters,
    "trainable_ratio": trainable_ratio,
}

def save_json(
    data: Any,
    file_path: Path,
) -> None:
    """
    파이썬 객체를 UTF-8 JSON 파일로 저장합니다.
    """

    # 부모 디렉터리가 존재하지 않으면 생성합니다.
    file_path.parent.mkdir(
        parents=True,
        exist_ok=True,
    )

    # 지정한 경로를 UTF-8 쓰기 모드로 엽니다.
    with file_path.open("w", encoding="utf-8") as file:

        # 데이터가 Tensor 등 JSON 변환이 어려운 값을 포함할 수 있으므로
        # default=str을 지정해 문자열 변환을 허용합니다.
        json.dump(
            data,
            file,
            ensure_ascii=False,

```

```

        indent=2,
        default=str,
    )

def run_test_inference(
    model,
    tokenizer,
    question: str,
) -> str:
    """
    현재 학습된 모델을 사용하여 한 개의 질문에 답변을 생성합니다.

    Args:
        model: 학습된 모델
        tokenizer: 모델 Tokenizer
        question: 사용자 질문

    Returns:
        생성된 답변 문자열
    """

    # 추론 시 KV Cache를 사용하여 생성 속도를 높입니다.
    model.config.use_cache = True

    # 모델을 평가 모드로 변경하여 Dropout을 비활성화합니다.
    model.eval()

    # 모델에 전달할 대화형 메시지를 구성합니다.
    messages = [
        {
            "role": "system",
            "content": (
                "당신은 온라인 쇼핑몰의 고객 상담 AI입니다. "
                "고객의 질문에 친절하고 정확하게 답변하세요."
            ),
        },
    ],

```

```

    {
        "role": "user",
        "content": question,
    },
]

# 모델 전용 Chat Template을 적용하여 대화를 하나의 문자열로 변환합니다.
prompt_text = tokenizer.apply_chat_template(
    messages,

    # 토큰 ID가 아니라 문자열을 반환받습니다.
    tokenize=False,

    # 모델이 assistant 답변을 생성하도록 생성 시작 토큰을 추가합니다.
    add_generation_prompt=True,
)

# 변환된 프롬프트를 PyTorch Tensor로 Tokenizing합니다.
inputs = tokenizer(
    prompt_text,
    return_tensors="pt",
)

# 입력 Tensor를 모델이 위치한 GPU로 이동합니다.
inputs = {
    key: value.to(model.device)
    for key, value in inputs.items()
}

# Gradient 계산을 비활성화하여 추론 메모리를 절약합니다.
with torch.inference_mode():

    # 모델로부터 새로운 토큰을 생성합니다.
    generated_ids = model.generate(
        **inputs,

        # 생성할 최대 신규 토큰 수입니다.

```

```

max_new_tokens=200,

# 샘플링 방식의 생성을 활성화합니다.
do_sample=True,

# 생성 결과의 무작위성을 조절합니다.
temperature=0.7,

# 누적 확률 상위 90% 토큰 중에서 샘플링합니다.
top_p=0.9,

# 반복 표현을 억제합니다.
repetition_penalty=1.1,

# Padding Token ID를 지정합니다.
pad_token_id=tokenizer.pad_token_id,

# EOS Token이 생성되면 답변 생성을 종료합니다.
eos_token_id=tokenizer.eos_token_id,
)

# 입력 프롬프트 부분을 제외하고 새로 생성된 토큰만 추출합니다.
new_tokens = generated_ids[
    :,
    inputs["input_ids"].shape[1]:
]

# 토큰 ID를 사람이 읽을 수 있는 문자열로 변환합니다.
answer = tokenizer.batch_decode(
    new_tokens,
    skip_special_tokens=True,
)[0]

# 앞뒤 공백을 제거한 답변을 반환합니다.
return answer.strip()

```

```

def main() -> None:
    """
    데이터 로드부터 모델 학습, 저장, 평가 및 추론까지
    전체 SFT 파이프라인을 실행합니다.
    """

    # 동일한 난수 흐름을 사용하도록 전역 Seed를 고정합니다.
    set_seed(SEED)

    # CUDA 연산 성능과 재현성 관련 설정을 적용합니다.
    if torch.cuda.is_available():

        # TensorFloat-32 행렬 곱셈을 허용하여
        # 지원 GPU에서 연산 속도를 향상시킬 수 있습니다.
        torch.backends.cuda.matmul.allow_tf32 = True

        # cuDNN에서도 TF32 연산을 허용합니다.
        torch.backends.cudnn.allow_tf32 = True

    # 현재 실행 환경을 출력합니다.
    print_environment()

    # 필요한 데이터 파일이 존재하는지 검사합니다.
    validate_files()

    # JSONL 데이터셋을 로드합니다.
    train_dataset, valid_dataset = load_sft_datasets()

    # 모델 Tokenizer를 로드합니다.
    tokenizer = load_tokenizer()

    # GPU 지원 여부에 따라 BF16 또는 FP16을 선택합니다.
    compute_dtype = select_compute_dtype()

    # 사전 학습 모델을 4bit 양자화 상태로 불러옵니다.
    model = load_quantized_model(
        tokenizer=tokenizer,

```

```

        compute_dtype=compute_dtype,
    )

    # LoRA Adapter 설정을 생성합니다.
    lora_config = create_lora_config()

    # SFT 학습 설정을 생성합니다.
    training_config = create_training_config(
        compute_dtype=compute_dtype,
    )

    # SFTTrainer를 생성합니다.
    # peft_config를 전달하면 Trainer가 모델에 LoRA Adapter를 적용합니다.
    trainer = SFTTrainer(
        # 학습할 사전 학습 모델입니다.
        model=model,

        # SFT 학습 설정입니다.
        args=training_config,

        # 학습용 대화 데이터셋입니다.
        train_dataset=train_dataset,

        # 검증용 대화 데이터셋입니다.
        eval_dataset=valid_dataset,

        # Tokenizing과 Chat Template 적용에 사용할 Tokenizer입니다.
        processing_class=tokenizer,

        # 모델에 추가할 LoRA Adapter 설정입니다.
        peft_config=lora_config,
    )

    # Trainer가 LoRA를 적용한 이후의 파라미터 개수를 계산합니다.
    parameter_info = count_parameters(trainer.model)

    # 파라미터 정보를 출력합니다.

```



```

print("\n" + "=" * 80)
print("6. 학습 파라미터 확인")
print("=" * 80)
print(
    f"전체 파라미터      : "
    f"{parameter_info['total_parameters'];,}"
)
print(
    f"학습 파라미터      : "
    f"{parameter_info['trainable_parameters'];,}"
)
print(
    f"학습 파라미터 비율: "
    f"{parameter_info['trainable_ratio']:.4f}%"
)

# Trainer가 제공하는 LoRA 파라미터 정보도 출력합니다.
if hasattr(trainer.model, "print_trainable_parameters"):
    trainer.model.print_trainable_parameters()

# 학습 시작 정보를 출력합니다.
print("\n" + "=" * 80)
print("7. Supervised Fine-Tuning 시작")
print("=" * 80)

# 실제 SFT 학습을 실행합니다.
train_result = trainer.train()

# 학습 완료 정보를 출력합니다.
print("\n" + "=" * 80)
print("8. 학습 완료")
print("=" * 80)
print(f"최종 Training Loss : {train_result.training_loss}")

# 학습 결과 Metric을 딕셔너리로 가져옵니다.
train_metrics = train_result.metrics

```

```
# 학습 데이터 수를 Metric에 추가합니다.
train_metrics["train_samples"] = len(train_dataset)

# 학습 결과를 Trainer 로그로 출력합니다.
trainer.log_metrics(
    "train",
    train_metrics,
)

# 학습 결과를 Trainer 형식으로 저장합니다.
trainer.save_metrics(
    "train",
    train_metrics,
)

# Trainer의 상태를 JSON 파일로 저장합니다.
trainer.save_state()

# 검증 데이터로 최종 평가를 수행합니다.
eval_metrics = trainer.evaluate()

# 검증 데이터 수를 Metric에 추가합니다.
eval_metrics["eval_samples"] = len(valid_dataset)

# 검증 결과를 화면에 출력합니다.
trainer.log_metrics(
    "eval",
    eval_metrics,
)

# 검증 결과를 파일로 저장합니다.
trainer.save_metrics(
    "eval",
    eval_metrics,
)

# 최종 LoRA Adapter를 지정된 출력 경로에 저장합니다.
```

```

trainer.save_model(str(OUTPUT_DIR))

# Tokenizer도 같은 경로에 저장합니다.
tokenizer.save_pretrained(str(OUTPUT_DIR))

# 학습에 사용된 주요 설정을 별도 JSON 파일로 저장합니다.
save_json(
    {
        "base_model": MODEL_NAME,
        "output_dir": str(OUTPUT_DIR),
        "max_length": MAX_LENGTH,
        "train_batch_size": TRAIN_BATCH_SIZE,
        "eval_batch_size": EVAL_BATCH_SIZE,
        "gradient_accumulation_steps": GRADIENT_ACCUMULATION_STEPS,
        "effective_batch_size": (
            TRAIN_BATCH_SIZE
            * GRADIENT_ACCUMULATION_STEPS
        ),
        "num_train_epochs": NUM_TRAIN_EPOCHS,
        "learning_rate": LEARNING_RATE,
        "weight_decay": WEIGHT_DECAY,
        "warmup_ratio": WARMUP_RATIO,
        "compute_dtype": str(compute_dtype),
        "lora_r": lora_config.r,
        "lora_alpha": lora_config.lora_alpha,
        "lora_dropout": lora_config.lora_dropout,
        "target_modules": list(lora_config.target_modules),
        "parameter_info": parameter_info,
        "train_metrics": train_metrics,
        "eval_metrics": eval_metrics,
    },
    OUTPUT_DIR / "training_summary.json",
)

# 학습된 모델로 간단한 질문을 테스트합니다.
test_question = "배송 완료로 나오지만 상품을 받지 못했습니다."

```

```

# 테스트 질문을 화면에 출력합니다.
print("\n" + "=" * 80)
print("9. 학습 모델 추론 테스트")
print("=" * 80)
print(f"[질문]\n{test_question}")

# 모델 답변을 생성합니다.
test_answer = run_test_inference(
    model=trainer.model,
    tokenizer=tokenizer,
    question=test_question,
)

# 생성 결과를 출력합니다.
print(f"\n[답변]\n{test_answer}")

# 테스트 결과를 JSON 파일로 저장합니다.
save_json(
    {
        "question": test_question,
        "answer": test_answer,
    },
    OUTPUT_DIR / "test_generation.json",
)

# 최종 저장 경로를 출력합니다.
print("\n" + "=" * 80)
print("10. 전체 파이프라인 완료")
print("=" * 80)
print(f"LoRA Adapter 저장 위치 : {OUTPUT_DIR}")
print(
    "다음 명령으로 저장된 Adapter를 불러와 추론할 수 있습니다.\n"
    "python 03_inference.py"
)

# Trainer와 모델 객체 참조를 제거합니다.
del trainer

```

```

del model

# 사용하지 않는 파이썬 객체를 정리합니다.
gc.collect()

# CUDA 캐시 메모리를 해제합니다.
if torch.cuda.is_available():
    torch.cuda.empty_cache()

# 이 파일을 직접 실행한 경우에만 main 함수를 호출합니다.
if __name__ == "__main__":
    main()

```

6. 저장된 LoRA Adapter 추론 코드

03_inference.py

```

"""
학습이 완료된 LoRA Adapter를 불러와
사용자 질문에 대한 답변을 생성하는 코드입니다.
"""

# os 모듈은 환경변수를 읽을 때 사용합니다.
import os

# Path는 파일과 디렉터리 경로를 처리합니다.
from pathlib import Path

# PyTorch는 GPU 연산과 Tensor 처리를 담당합니다.
import torch

# .env 파일의 환경변수를 불러옵니다.
from dotenv import load_dotenv

```

```

# 사전 학습 모델, Tokenizer 및 양자화 설정을 불러옵니다.
from transformers import (
    AutoModelForCausalLM,
    AutoTokenizer,
    BitsAndBytesConfig,
)

# 기본 모델에 학습된 LoRA Adapter를 연결하기 위해 사용합니다.
from peft import PeftModel

# 현재 파일의 위치를 프로젝트 기준 디렉터리로 지정합니다.
BASE_DIR = Path(__file__).resolve().parent

# .env 환경변수를 읽습니다.
load_dotenv(BASE_DIR / ".env")

# 학습에 사용한 기본 모델 이름을 가져옵니다.
BASE_MODEL_NAME = os.getenv(
    "MODEL_NAME",
    "Qwen/Qwen2.5-0.5B-Instruct",
)

# 학습된 LoRA Adapter 디렉터를 설정합니다.
ADAPTER_DIR = BASE_DIR / os.getenv(
    "OUTPUT_DIR",
    "outputs/qwen2.5-korean-sft-lora",
)

# Hugging Face 접근 토큰을 읽습니다.
HF_TOKEN = os.getenv("HF_TOKEN") or None

def select_compute_dtype() -> torch.dtype:
    """
    GPU가 BF16을 지원하면 BF16을 사용하고,
    그렇지 않으면 FP16을 사용합니다.

```

```

"""

# CUDA GPU가 BF16을 지원하는지 확인합니다.
if (
    torch.cuda.is_available()
    and torch.cuda.is_bf16_supported()
):
    return torch.bfloat16

# BF16을 지원하지 않으면 FP16을 반환합니다.
return torch.float16

def load_model_and_tokenizer():
    """
    기본 모델과 학습된 LoRA Adapter 및 Tokenizer를 불러옵니다.

    Returns:
        model: Adapter가 연결된 추론 모델
        tokenizer: 학습 시 저장한 Tokenizer
    """

    # 학습된 Adapter 디렉터리가 존재하는지 검사합니다.
    if not ADAPTER_DIR.exists():
        raise FileNotFoundError(
            f"LoRA Adapter를 찾을 수 없습니다: {ADAPTER_DIR}\n"
            "먼저 python 02_train_sft.py를 실행하세요."
        )

    # GPU 연산에 사용할 데이터 타입을 선택합니다.
    compute_dtype = select_compute_dtype()

    # 4bit 양자화 설정을 생성합니다.
    quantization_config = BitsAndBytesConfig(

        # 기본 모델을 4bit 상태로 로드합니다.
        load_in_4bit=True,

```

```

# NF4 양자화 방식을 사용합니다.
bnb_4bit_quant_type="nf4",

# 이중 양자화를 적용합니다.
bnb_4bit_use_double_quant=True,

# 실제 계산은 BF16 또는 FP16으로 수행합니다.
bnb_4bit_compute_dtype=compute_dtype,
)

# 학습할 때 저장한 Tokenizer를 Adapter 경로에서 불러옵니다.
tokenizer = AutoTokenizer.from_pretrained(
    str(ADAPTER_DIR),
    trust_remote_code=True,
)

# Padding Token이 없으면 EOS Token을 사용합니다.
if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token

# 기본 사전 학습 모델을 4bit로 불러옵니다.
base_model = AutoModelForCausalLM.from_pretrained(
    BASE_MODEL_NAME,
    quantization_config=quantization_config,
    device_map={"": 0},
    trust_remote_code=True,
    token=HF_TOKEN,
)

# 기본 모델의 Padding Token ID를 설정합니다.
base_model.config.pad_token_id = tokenizer.pad_token_id

# 기본 모델 위에 학습된 LoRA Adapter를 연결합니다.
model = PeftModel.from_pretrained(
    base_model,
    str(ADAPTER_DIR),

```



```

)

# 추론 속도 향상을 위해 KV Cache를 활성화합니다.
model.config.use_cache = True

# 모델을 평가 모드로 변경합니다.
model.eval()

# 완성된 모델과 Tokenizer를 반환합니다.
return model, tokenizer

def generate_answer(
    model,
    tokenizer,
    question: str,
) -> str:
    """
    사용자 질문을 모델에 입력하여 답변을 생성합니다.

    Args:
        model: LoRA Adapter가 적용된 모델
        tokenizer: 모델 Tokenizer
        question: 사용자가 입력한 질문

    Returns:
        생성된 답변 문자열
    """

    # 모델에게 전달할 대화 메시지를 구성합니다.
    messages = [
        {
            "role": "system",
            "content": (
                "당신은 온라인 쇼핑몰의 고객 상담 AI입니다. "
                "고객의 질문을 정확히 이해하고 친절하게 답변하세요. "
                "확인되지 않은 사실은 임의로 만들지 마세요."
            )
        }
    ]

```

```

        ),
    },
    {
        "role": "user",
        "content": question,
    },
]

# Qwen 모델의 Chat Template을 적용합니다.
prompt = tokenizer.apply_chat_template(
    messages,
    tokenize=False,
    add_generation_prompt=True,
)

# 프롬프트를 Token ID와 Attention Mask로 변환합니다.
inputs = tokenizer(
    prompt,
    return_tensors="pt",
)

# 입력 데이터를 모델이 위치한 GPU로 이동합니다.
inputs = {
    key: value.to(model.device)
    for key, value in inputs.items()
}

# Gradient 계산이 필요하지 않은 추론 모드로 실행합니다.
with torch.inference_mode():

    # 모델 답변 토큰을 생성합니다.
    generated_ids = model.generate(
        **inputs,
        max_new_tokens=256,
        do_sample=True,
        temperature=0.6,
        top_p=0.9,
    )

```

```

        repetition_penalty=1.1,
        pad_token_id=tokenizer.pad_token_id,
        eos_token_id=tokenizer.eos_token_id,
    )

    # 입력 프롬프트 부분을 제외한 신규 생성 토큰만 추출합니다.
    generated_tokens = generated_ids[
        :,
        inputs["input_ids"].shape[1]:
    ]

    # 생성된 토큰을 문자열로 변환합니다.
    answer = tokenizer.batch_decode(
        generated_tokens,
        skip_special_tokens=True,
    )[0]

    # 답변의 앞뒤 공백을 제거하여 반환합니다.
    return answer.strip()

```

def main() -> None:

```

"""
모델을 한 번 로드한 후 사용자의 질문을 반복해서 처리합니다.
"""

# 프로그램 제목을 출력합니다.
print("=" * 70)
print("Qwen2.5 한국어 고객 상담 SFT 모델")
print("=" * 70)

# 기본 모델과 LoRA Adapter 및 Tokenizer를 불러옵니다.
model, tokenizer = load_model_and_tokenizer()

# 모델 로드 완료 메시지를 출력합니다.
print("모델과 LoRA Adapter 로드가 완료되었습니다.")
print("'종료', 'exit' 또는 'quit'을 입력하면 종료됩니다.")

```

```

# 사용자가 종료 명령을 입력할 때까지 반복합니다.
while True:

    # 사용자 질문을 입력받습니다.
    question = input("\n질문: ").strip()

    # 종료 명령어 목록에 포함되면 반복을 종료합니다.
    if question.lower() in {
        "종료",
        "exit",
        "quit",
    }:
        print("프로그램을 종료합니다.")
        break

    # 아무 내용도 입력하지 않은 경우 다시 입력받습니다.
    if not question:
        print("질문을 입력해 주세요.")
        continue

    # 모델을 사용해 답변을 생성합니다.
    answer = generate_answer(
        model=model,
        tokenizer=tokenizer,
        question=question,
    )

    # 생성된 답변을 출력합니다.
    print(f"\n답변: {answer}")

# 이 파일을 직접 실행한 경우 main 함수를 호출합니다.
if __name__ == "__main__":
    main()

```

7. 실행 순서

7.1 가상환경 생성

Windows PowerShell 또는 PyCharm Terminal에서 실행합니다.

```
python -m venv .venv
```

가상환경을 활성화합니다.

```
.venv\Scripts\activate
```

RunPod 또는 Linux에서는 다음 명령을 사용합니다.

```
python -m venv .venv
```

```
source .venv/bin/activate
```

7.2 라이브러리 설치

```
python -m pip install --upgrade pip setuptools wheel
```

```
pip install -r requirements.txt
```

7.3 학습 데이터 생성

```
python 01_create_dataset.py
```

정상 실행 결과는 다음과 비슷합니다.

```
=====
SFT 데이터셋 생성 완료
=====
전체 데이터 수 : 20
학습 데이터 수 : 16
검증 데이터 수 : 4
학습 파일      : ../data/train.jsonl
검증 파일      : ../data/valid.jsonl
```

7.4 모델 학습

```
python 02_train_sft.py
```

학습 중에는 다음 값이 출력됩니다.

loss
grad_norm
learning_rate
epoch
eval_loss
train_runtime
train_samples_per_second

학습 완료 후 다음 파일들이 생성됩니다.

outputs/qwen2.5-korean-sft-lora/
├── adapter_config.json
├── adapter_model.safetensors
├── tokenizer.json
├── tokenizer_config.json
├── special_tokens_map.json
├── training_summary.json
└── test_generation.json

adapter_model.safetensors는 전체 Qwen 모델이 아니라 학습된 LoRA 파라미터만 저장한 파일입니다.

7.5 저장된 모델 추론

python 03_inference.py

실행 예시는 다음과 같습니다.

질문: 배송 완료로 표시되지만 상품을 받지 못했습니다.

답변: 먼저 가족, 경비실, 무인 택배함 또는 문 앞에 상품이 보관되었는지 확인해 주세요. 상품이 확인되지 않는 경우 배송 조회에 표시된 택배사에 문의한 뒤 주문번호와 함께 고객센터로 문의해 주세요.

8. TensorBoard로 학습 Loss 확인

학습이 완료된 후 프로젝트 터미널에서 실행합니다.

```
tensorboard --logdir outputs/checkpoints/runs
```

RunPod에서는 다음과 같이 외부 접속을 허용합니다.

```
tensorboard ₩
  --logdir outputs/checkpoints/runs ₩
  --host 0.0.0.0 ₩
  --port 6006
```

그다음 RunPod의 HTTP Service에서 6006 포트를 연결합니다.

9. GPU 메모리 부족 시 수정할 설정

CUDA out of memory 오류가 발생하면 02_train_sft.py에서 다음 순서로 조정합니다.

```
# 입력 길이를 512에서 256으로 줄입니다.
MAX_LENGTH = 256

# 실제 GPU 배치 크기는 1로 유지합니다.
TRAIN_BATCH_SIZE = 1

# Gradient 누적 횟수를 늘려 유효 배치 크기를 확보합니다.
GRADIENT_ACCUMULATION_STEPS = 16

# LoRA Rank를 16에서 8로 줄입니다.
r = 8

# LoRA 적용 범위를 Attention Layer 중심으로 줄입니다.
target_modules = [
    "q_proj",
    "k_proj",
    "v_proj",
    "o_proj",
]
```

10. 실습 성공 확인 기준

다음 항목이 모두 확인되면 SFT 전체 파이프라인이 정상적으로 완료된 것입니다.

1. CUDA GPU가 정상적으로 인식된다.
2. Qwen 기본 모델과 Tokenizer가 다운로드된다.
3. train.jsonl과 valid.jsonl이 로드된다.
4. LoRA 학습 가능 파라미터 수가 출력된다.
5. 학습 과정에서 Loss가 출력된다.
6. 검증 데이터의 eval_loss가 출력된다.
7. adapter_model.safetensors가 저장된다.
8. 저장된 Adapter를 다시 불러올 수 있다.
9. 사용자 질문에 대해 답변이 생성된다.
10. training_summary.json과 test_generation.json이 저장된다.

이 실습의 1차 성공 기준은 답변 품질 자체보다 **데이터 준비 → 모델 로드 → SFT 학습 → Adapter 저장 → 재로드 → 추론까지의 전체 과정이 중단 없이 실행되는 것**입니다.